

1)Introducción a Ing de Sw

a)Software

Software: es un set de programas y la documentación que acompaña. Existen tres tipos básicos de software: System software ▪ Utilitarios ▪ Software de Aplicación

El software es conocimiento representado en distintos niveles de abstracción.

Modelos aristotélico del software:

- El Accidente (La Representación): Dificultades que acompañan la producción, pero no son inherentes. (Ej: Sintaxis compleja, limitaciones de memoria, tiempos de compilación). Ya casi los resolvimos todos.
- La Esencia (El Concepto): La construcción de estructuras conceptuales entrelazadas altamente complejas: algoritmos, flujos de datos, relaciones. Esto es lo que queda hoy.
- Escribir código no es el problema real. La parte difícil es la especificación, el diseño y la validación de la estructura conceptual.

Propiedades irreducibles del software complejo: La esencia del software resiste la automatización porque posee características únicas en la ingeniería humana.

- 1. Complejidad: Escalar software aumenta los elementos interactuando de forma no lineal. No hay piezas repetidas a nivel macro.
- 2. Conformidad: Debe adaptarse a interfaces humanas arbitrarias y sistemas sin principios unificadores, dictados por el entorno.
- 3. Mutabilidad: Es pura "materia de pensamiento", infinitamente maleable y constantemente sometida a la presión del cambio continuo.
- 4. Invisibilidad: El software carece de representación espacial o geométrica. No puede dibujarse en un plano. Priva a la mente de la abstracción visual.

El cuello de botella humano: Decidir el "Qué" "La parte más difícil de construir software no es decirlo, sino decidir qué decir."

La idea del producto (mental) → El Prompt El nuevo lenguaje de alto nivel. Para que la IA funcione, el humano debe especificar con precisión matemática restricciones, estados y lógicas de negocio. → El límite fundamental: Si el humano no comprende la complejidad esencial, el prompt será incompleto. La IA no deduce la intención no escrita. La programación automática históricamente solo funcionó en dominios con parámetros altamente restringidos.

b)Ingeniería de Software

Ingeniería de software: La ingeniería de software es una disciplina de la ingeniería que se ocupa de todos los aspectos de la producción de software, desde la concepción inicial del sistema hasta su operación y mantenimiento

Estado actual y antecedentes: La crisis del software

La crisis del software hace referencia a los problemas que surgieron con el desarrollo de software, planteados en la conferencia de OTAN de 1968 por Friedrich Bauer. Se destacó la dificultad de crear software libre de defectos, comprensible y verificable. El término también había sido utilizado anteriormente por Dijkstra.

Causas:

- El avance del hardware permitió desarrollar sistemas cada vez más grandes y complejos, pero el desarrollo del software no avanzó al mismo ritmo.
- Aumentó la demanda de sistemas complejos y se subestimó la dificultad de desarrollarlos.
- Los productos debían adaptarse constantemente a las necesidades del cliente.
- Faltaba una disciplina que interviniera en todos los aspectos de la producción del software.

En la actualidad, muchos sistemas siguen fracasando principalmente por:

- Demandas crecientes: se necesitan sistemas más grandes y complejos, desarrollados rápidamente, pero los métodos tradicionales no siempre permiten hacerlo.
- Bajas expectativas: muchas empresas no aplican métodos de ingeniería de software, generando software más costoso y menos confiable. Además, se conforman con que simplemente “funcione”, sin considerar el valor que aporta al negocio.

Ingeniería y calidad de software; la materia en contexto

- **Disciplinas técnicas:** nos referimos a actividades que aportan al desarrollo de software como producto.
 - Requerimientos
 - Análisis y Diseño
 - Construcción
 - Prueba
 - Despliegue
- **Disciplinas de Gestión:** hacen referencia a actividades de planificación, monitoreo y control del proyecto de desarrollo de software. Se utilizan, por ejemplo, metodologías LEAN Agile de gestión de proyectos.
 - Planificación de Proyecto
 - Monitoreo y Control de Proyectos
- **Disciplinas de Soporte:** son disciplinas transversales a los procesos de software, que permiten verificar la integridad y calidad de un producto de software. Entre ellas se incluyen
 - Gestión de Configuración de Software
 - Aseguramiento de Calidad
 - Métricas

Ejemplos de grandes proyectos de software fallidos y exitosos.

- Fallidos
 - **Steam Machine:** consola basada en Linux que buscaba llevar los juegos de PC a las consolas. Fracásó, entre otras cosas, por retrasos, cancelaciones y alto costo.
 - **Unity / Ubuntu Phone:** Canonical apostó por Unity, Ubuntu Touch y Mir, pero los problemas de desarrollo y la falta de innovación llevaron a abandonar estos proyectos y pasar a GNOME.
 - **Ubuntu Edge:** teléfono que buscaba lograr la **convergencia** entre móvil y escritorio. Intentó financiarse mediante crowdfunding, pero no alcanzó el objetivo y nunca llegó a producirse.

- Exitosos:
 - **GNU/Linux:** Sistema operativo formado por el núcleo Linux y las herramientas de GNU. Su éxito se debe a que es **gratuito, completo, configurable y multiplataforma**, y se utiliza en servidores, supercomputadoras y dispositivos.
 - **2. Mozilla Firefox:** Navegador que ayudó a cambiar el dominio de Internet Explorer. Se destacó por ser **rápido, seguro, multiplataforma y respetuoso de los estándares web**, impulsando la competencia entre navegadores.
 - **3. Apache:** Servidor web utilizado para proporcionar páginas web a los usuarios. Su **versatilidad y posibilidad de modificación** gracias al software libre lo convirtieron en uno de los servidores web más utilizados.

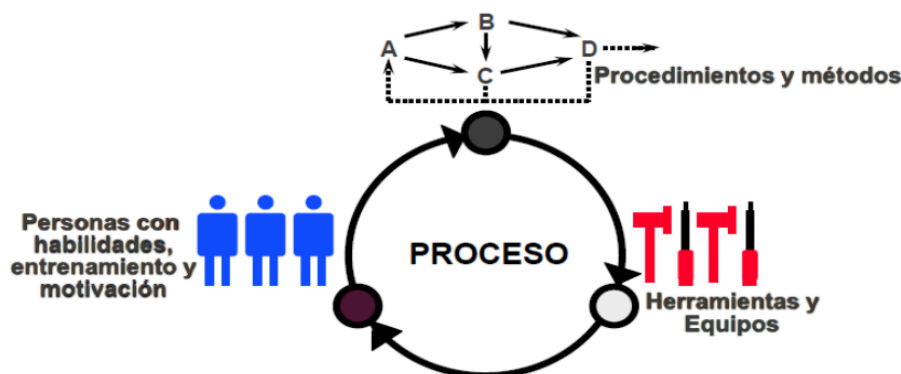
c)Proceso de software



Proceso: La secuencia de pasos ejecutados para un propósito dado (IEEE)

Proceso de software: Un conjunto de actividades, métodos, prácticas, y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados (Sw-CMM)

factores determinantes del proceso:



- **Procedimientos y Métodos:** La adaptación de los procesos establecidos para su implementación, dependiendo del contexto, es esencial para la calidad resultante. Definir las actividades que se harán y cuáles no.
- **Personas motivadas, capacitadas y con habilidades:** Es factor primordial para obtener la calidad del producto del proceso, ya que éste se determina del esfuerzo de las personas. Sin ellas, no se puede lograr construir ningún producto.
- **Herramientas y Equipos:** Materiales necesarios para llevar a cabo el proceso que nos permiten que las entradas se transformen en salidas. Se recomienda automatizar la mayor cantidad de actividades posibles, lo que mejora la eficiencia de las personas y puedan concentrarse en tareas que requieran de su capacidad.

Tipos de procesos

Procesos definidos: (inspirados en las líneas de producción) Asume que podemos repetir el mismo proceso una y otra vez, indefinidamente, y obtener los mismos resultados. La administración y control provienen de la predictibilidad del proceso definido.

Los procesos definidos son considerados deterministas: ante la misma entrada se pretende que se obtendrá la misma salida. Asumen que, si se aplican una y otra vez, se obtendrán siempre los mismos resultados, a pesar de cambiar de equipo.



Procesos Empíricos:

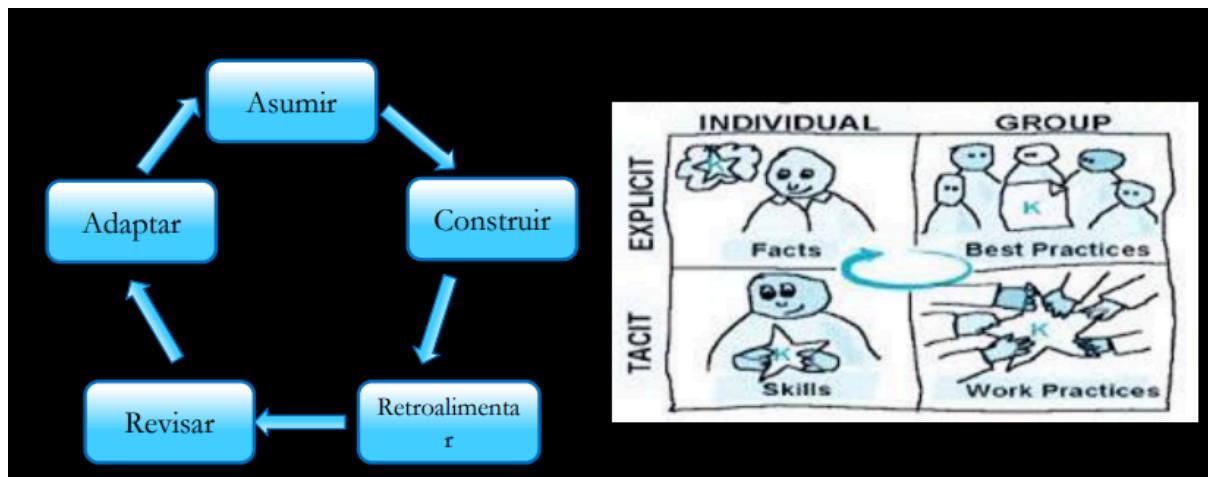
Asume procesos complicados con variables cambiantes. Cuando se repite el proceso, se pueden llegar a obtener resultados diferentes.

La administración y control es a través de inspecciones frecuentes y adaptaciones. Son procesos que trabajan bien con procesos creativos y complejos.

Los procesos empíricos se conforman en base a la experiencia interna y externa de las personas que intervienen en un contexto particular

1. Se basan en ciclos cortos de inspección y adaptación (retroalimentación), porque ante un análisis, se ajustan de mejor forma en aquellos dominios complejos donde prima la creatividad y/o complejidad.
2. La administración y el control es por medio de inspecciones frecuentes y adaptaciones para lograr buenas prácticas.
3. No se pueden combinar con cualquiera de los tipos de ciclos de vida vistos, se suele recomendar que sea el modelo iterativo-incremental, y el que se suele prohibir es el modelo secuencial debido a que no es viable después de 2 años (lo dice una teoría).

Patrón de conocimiento:

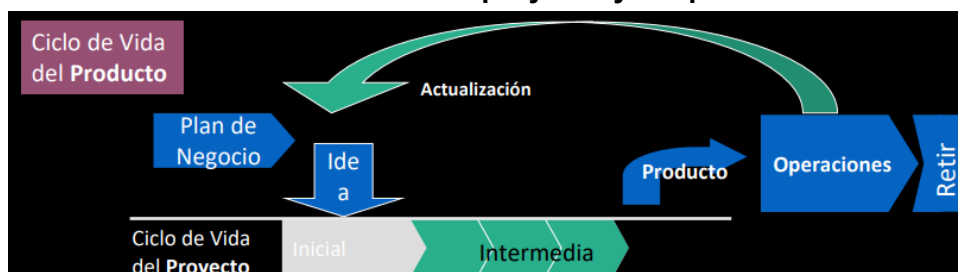


La experiencia que se obtiene en cada uno de los procesos no es extrapolable, es decir, que solo funciona en ese mismo proyecto, y se necesita adaptar los procesos a cada contexto en especial. Los conocimientos obtenidos en el proceso se deben hacer explícitos al grupo de desarrollo, y convertir las habilidades individuales en buenas prácticas del trabajo en grupo (compartir conocimiento). Además de la inspección y adaptación, los procesos empíricos poseen un tercer pilar muy importante, que es la transparencia de la información ante todos los integrantes del equipo de trabajo.

d) ciclos de vida

Ciclos de vida: La serie de pasos a través de los cuales el producto o proyecto progresa. Los productos tienen su ciclo de vida. Los proyectos también.

Relación entre el ciclo de vida del proyecto y del producto:



No existe un impacto entre los ciclos de vida de cada uno, sino que el ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto, ya que el ciclo de vida del proyecto dura lo que dura el desarrollo del software. En cambio, el ciclo de vida del producto dura hasta que el software se deje de utilizar, dependiendo esto de la madurez que tenga el producto en base a las necesidades del mercado.

Es posible que un producto tenga varios proyectos en su ciclo de vida, debido a, constantes cambios y/o actualizaciones que se vayan realizando. Por lo que, dentro del ciclo de vida del producto, se pueden desarrollar varios ciclos de vida de proyectos.

Ciclos de Vida de proyectos de software

- Un ciclo de vida de un proyecto software es una representación de un proceso. Gráfica una descripción del proceso desde una perspectiva particular
- Los modelos especifican
 - Las fases de proceso. Ejemplo: requerimientos, especificación y diseño...
 - El orden en el cual se llevan a cabo

Clasificación de los ciclos de vida:

Hay tres tipos básicos de Ciclos de Vida

- **Secuencial:** Este modelo dispone de las actividades de forma lineal, es decir, que el proyecto progresa a través de una secuencia ordenada de pasos (fases) y una actividad no puede iniciar sin que la precedente haya sido finalizada.
 - **Ventajas:**
 - Permite planificar todo el proyecto desde el inicio.
 - Es sencillo de gestionar y controlar.
 - Funciona bien cuando los requerimientos son claros y estables.
 - Genera documentación durante todo el proyecto.
 - **Desventajas:**
 - Es difícil adaptarse a cambios en los requerimientos.
 - El cliente ve el producto funcional recién al final.
 - Los errores detectados tarde pueden requerir mucho retrabajo.
 - Las etapas dependen unas de otras.
 - Puede generar demasiada documentación y burocracia.
- **Iterativo/Incremental:** Este modelo aplica sucesivas iteraciones en forma escalonada a medida que avanza el calendario de actividades. Cada iteración produce un incremento de software funcional potencialmente entregable. Este ciclo de vida se utiliza en metodologías ágiles.
 - **Ventajas:**
 - Permite adaptarse a cambios en los requerimientos.
 - El cliente participa y da retroalimentación durante el desarrollo.
 - Se pueden entregar versiones funcionales más rápidamente.
 - Permite obtener valor del software antes de terminar todo el sistema.
 - Es útil cuando los requerimientos cambian con frecuencia.
 - **Desventajas:**
 - El proceso puede ser menos visible o difícil de controlar.
 - Documentar cada incremento puede tener un costo alto.
 - La estructura del software puede degradarse si se realizan muchos incrementos.
- **Recursoivo :**El modelo recursivo es utilizado para gestionar los riesgos del desarrollo de sistemas complejos a gran escala. Requiere de la intervención del cliente.
 - **Ventajas:**
 - Permite identificar y reducir riesgos en proyectos complejos.
 - El producto se mejora en cada iteración.
 - Requiere la participación del cliente.
 - Permite generar componentes que pueden ser reutilizables.
 - **Desventajas:**
 - Puede ser costoso en tiempo y dinero.
 - La tecnología utilizada puede quedar obsoleta.
 - La falta de mantenimiento o documentación puede dificultar la reutilización.
 - Se obtiene una versión del producto recién al final.

Criterios para la selección de un modelo de proceso

Es necesario evaluar los niveles de riesgo, la claridad y cantidad de requerimientos, los conocimientos técnicos con los que cuenta el equipo de desarrollo, los plazos que se tienen para ejecutar el proyecto y la posibilidad de poder validar los resultados del trabajo con el cliente, también el trabajo en equipo.

1. Si los requerimientos son claros y exhaustivos, en un contexto de baja incertidumbre en el que el cliente no requiere obtener el producto rápidamente, es posible aplicar el modelo secuencial.
2. Si la situación anterior no se presenta y el cliente desea obtener resultados lo antes posible, con las funciones principales del sistema, entonces sería conveniente la elección del modelo iterativo- incremental.
3. Si el objetivo es disminuir los riesgos presentes en el desarrollo de sistemas grandes y complejos, entonces el modelo recursivo puede ser una buena opción.

Ciclo de Vida	Procesos de desarrollo que lo admite	Características que lo hacen elegible
Secuencial	Definidos	• Alta certeza (baja incertidumbre). • La organización tiene una forma tradicional de trabajar. • No se pueden realizar entregas parciales del software (se debe entregar todo junto). • Eliminar riesgos de planificación.
Iterativo/incr emental	Definidos o Empíricos	• Existe incertidumbre. • Volatilidad de requerimientos. • Posibilidad de entregas parciales. • Uso en etapas tempranas del producto.
Rekursivo	Definidos	• Mucho control de riesgos en cada iteración. • No se pueden realizar entregas parciales.

2)Software Configuration Management (SCM)

Software en contexto



Cambios en el software:

Tienen su origen en:

- ❖ Cambios del negocio y nuevos requerimientos
- ❖ Soporte de cambios de productos asociados

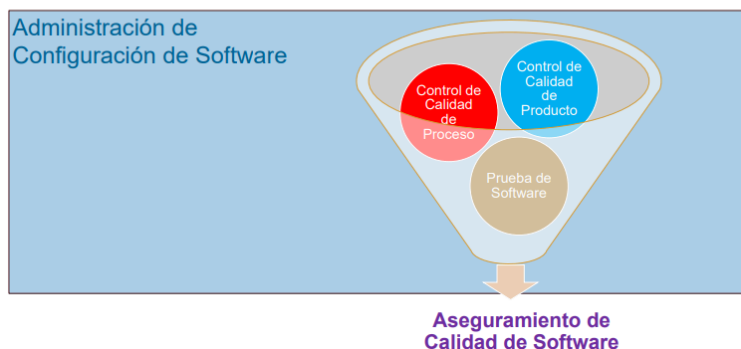
- ❖ Reorganización de las prioridades de la empresa por crecimiento
- ❖ Cambios en el presupuesto
- ❖ Defectos encontrados a corregir
- ❖ Oportunidades de mejora

Los sistemas de software siempre cambian durante su desarrollo y uso. Conforme se hacen cambios al software, se crean nuevas versiones del sistema. Los sistemas pueden considerarse como un conjunto de versiones donde cada una de ellas debe mantenerse y gestionarse. Esto es necesario para no perder la trazabilidad de los cambios que se incorporan a cada versión.

a)SCM como disciplina de soporte

Es una actividad “paraguas”, transversal a todo el proyecto, relevante para el producto a lo largo de su ciclo de vida

Disciplinas de soporte del Software



Historia

Tiene su origen a mediados de 1950s, cuando CM (por Configuration Management) originalmente utilizado para desarrollo de hardware y control de producción, fue utilizado en el desarrollo de software.

b)Definición SCM

Una disciplina que aplica dirección y monitoreo administrativo y técnico a: identificar y documentar las características funcionales y técnicas de los ítems de configuración, controlar los cambios de esas características, registrar y reportar los cambios y su estado de implementación y verificar correspondencia con los requerimientos (ANSI/IEEE 828, 1990)

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida. Esto implica identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su integridad y origen.

Decimos que se mantiene la integridad de un producto de software cuando satisface las necesidades del usuario, puede ser fácil y completamente rastreado durante su ciclo de vida, satisface criterios de performance, cumple con sus expectativas de costo

problemas del manejo de componentes

- Pérdida de un componente
- Pérdida de cambios (el componente que tengo no es el último)
- Sincronía fuente - objeto – ejecutable
- Regresión de fallas
- Doble mantenimiento
- Superposición de cambios
- Cambios no validados

El software es muy fácil de modificar, en el sentido de que uno puede entrar al repositorio y con un solo click eliminar meses de trabajo. Entonces lo que buscamos con la gestión de configuración del software es tratar de que estas cosas no pasen, buscando que el producto de software sea íntegro y si fuera el caso de que se van a realizar cambios, podamos tener un control de qué cosas cambiaron en un determinado momento. Es por esta razón que se asocia el concepto de software con el concepto de versión, ya que cada ítem de configuración debe tener su versión. Y todas las personas se enteren de los cambios.

Conceptos clave para la gestión de configuración de software

- Ítem de Configuración
- Repositorio
- Línea Base
- Ramas (Branch)
- Configuración del Software

c)Definiciones clave

-Ítem de Configuración de Software (SCI)

Se llama ítem de configuración (IC) a todos y cada uno de los artefactos que forman parte del producto o del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo y sobre los cuales necesitamos conocer su estado y evolución

- Documentos de diseño, código fuente, código ejecutable, etc
- Ejemplos:
 - Plan de CM ❖ Propuestas de Cambio ❖ Visión ❖ Riesgos ❖ Plan de desarrollo ❖ Prototipo de Interfaz ❖ Guía de Estilo de IHM ❖ Manual de Usuario ❖ Requerimientos ❖ Plan de Calidad ❖ Arquitectura del Software ❖ Plan de Integración ❖ Planes de Iteración ❖ Estándares de codificación ❖ Casos de prueba ❖ Código fuente ❖ Gráficos, iconos, ... ❖ Instructivo de ensamble ❖ Programa de instalación ❖ Documento de despliegue ❖ Lista de Control de entrega ❖ Formulario de aceptación ❖ Registro del proyecto

Versión: Una versión se define, desde el punto de vista de la evolución, como la forma particular de un artefacto en un instante o contexto dado.

El control de versiones se refiere a la evolución de un único ítem de configuración (IC), o de cada IC por separado.

La evolución puede representarse gráficamente en forma de grafo.

Variante: Una variante es una versión de un ítem de configuración (o de la configuración) que evoluciona por separado.

Las variantes representan configuraciones alternativas.

Un producto de software puede adoptar distintas formas (configuraciones) dependiendo del lugar donde se instale.

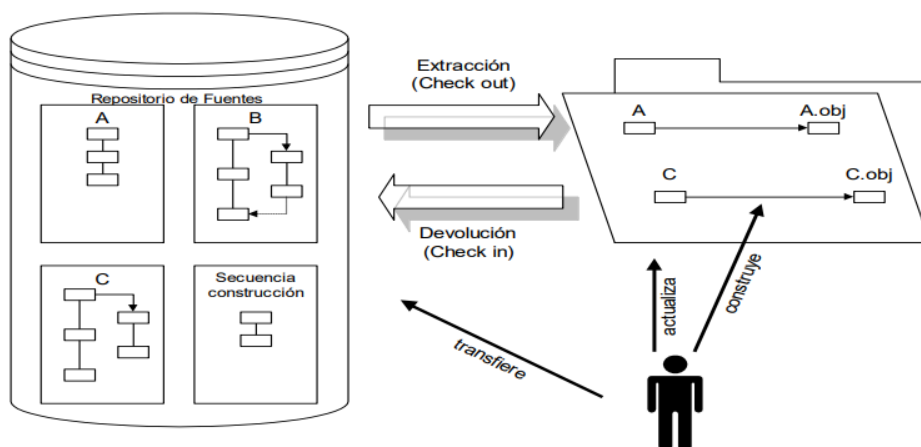
Por ejemplo, dependiendo de la plataforma (máquina + S.O.) que la soporta, o de las funciones opcionales que haya de realizar o no.

Entonces: La Configuración del Software es un conjunto de ítems de configuración con su correspondiente versión en un momento determinado

-Repositorio

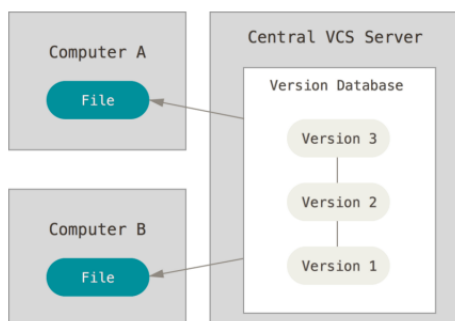
Es un contenedor de ítems de configuración, se encarga de mantener la historia de cada ítem con sus atributos y relaciones, además es usado para hacer evaluaciones de impacto de los cambios propuestos. Puede ser una o varias bases de datos. Tiene una estructura para mantener el orden y la integridad.

funcionamiento:



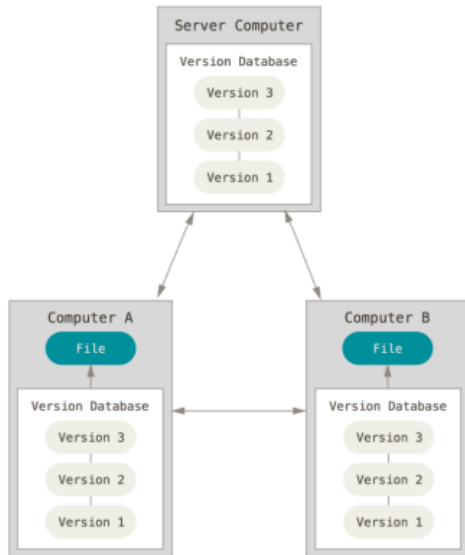
Repositorios centralizados

está caracterizado porque un servidor contiene todos los archivos con sus versiones. Los administradores tienen un mayor control sobre el repositorio. Tiene como desventaja que si falla el servidor, todo lo positivo se cae.



Repositorios Descentralizados

está caracterizado porque cada cliente tiene una copia exactamente igual del repositorio completo. Tiene como ventaja que se resuelve el problema de los servidores centralizados, debido a que si el servidor falla sólo es cuestión de “copiar y pegar”, además posibilita otros workflows no disponibles en el modelo centralizado. La desventaja que podemos mencionar es que es más complicado llevar un control sobre el mismo.



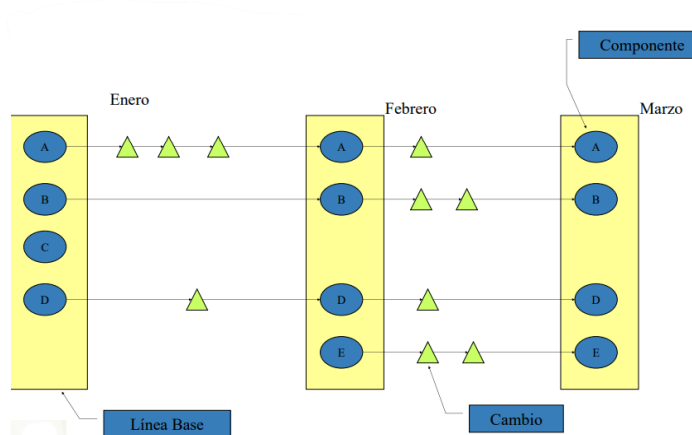
-Identificación de la Línea Base

Línea base : Una configuración que ha sido revisada formalmente y sobre la que se ha llegado a un acuerdo

- Sirve como base para desarrollos posteriores y puede cambiarse sólo a través de un procedimiento formal de control de cambios
- Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto

Este conjunto de ítems debe tener una referencia única, y esto se hace a través de “tags”.
Cómo la identificamos:

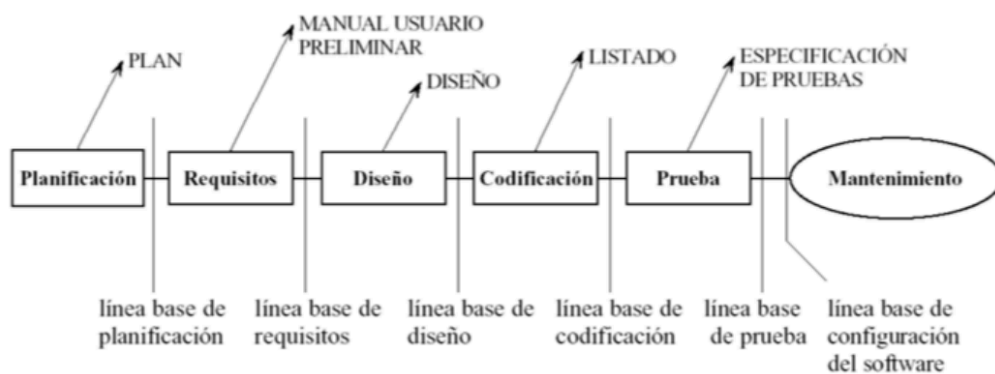
- ❖ Se utilizan etiquetas para “marcar” las baseline
- ❖ No confundir con la versión del Producto



Representación de Líneas Base

Pueden ser:

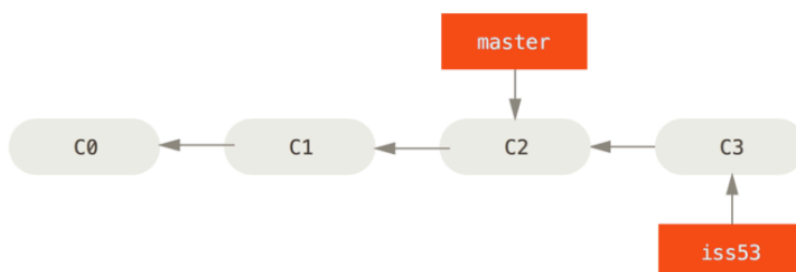
- De especificación (Requerimientos, Diseño): son las primeras línea base, dado que no cuentan con código. Podría contener el documento de especificación de requerimiento. Son de requerimientos, diseño.
- De productos: contiene una versión de producto cuyo código es ejecutable, han pasado por un control de calidad definido previamente. La primera línea base operacional corresponde con la primera Release. Es la línea base de productos que han pasado por un control de calidad definido previamente.



-Ramas

- Existe una rama principal (trunk, master)
- Sirven para bifurcar el desarrollo
- Pueden tener razones de creación con semántica
- Permiten la experimentación
- Pueden ser descartadas o integradas

Es un conjunto de ítems de configuración con sus correspondientes versiones, que permiten bifurcar el desarrollo de un software, por varios motivos: Experimentación; Resolución de errores en el desarrollo; Desarrollar un mismo software para diferentes plataformas.



Integración de ramas

- ❖ La operación se llama merge
- ❖ Lleva los cambios a la rama principal
- ❖ Pueden surgir conflictos (resolvemos con diff)

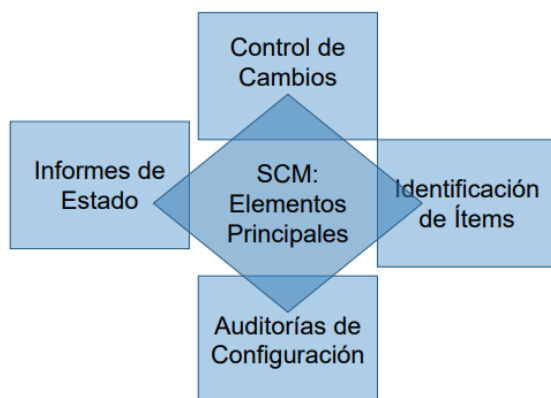
❖ Todas las ramas deberían eventualmente integrarse a la principal o ser descartadas. Se da cuando se integran dos ramas, para fusionar la configuración de los ítems de configuración con sus correspondientes versiones, en cada una de ellas. Una buena práctica es mantener la rama principal como la versión estable, y fusionar los cambios hacia ella. En caso de no integrarse a la Main, las ramas deberían descartarse.

-Configuración

Conjunto de todos los ítems de configuración con su versión específica. Define una foto de los ítems de configuración en un momento de tiempo determinado. Una configuración es el conjunto de todos los componentes fuentes que son compilados, sus documentos y la información de la estructura que definen una versión del producto a entregar. La configuración de un software es la sumatoria de todos los ítems de configuración que tiene en un momento determinado, equivale a una instantánea o una foto de todos los ítems de configuración con su versión en un momento del tiempo.

d)Actividades Fundamentales de la Gestión de Configuración de Software

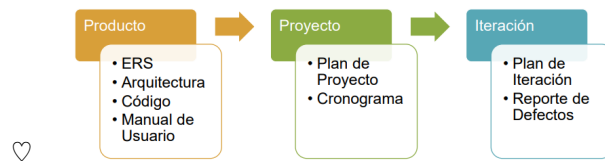
Estas actividades/elementos son las cosas que contienen las secciones de un plan de gestión de configuración. Recordemos que la gestión de configuración también se debe planificar. Todas las actividades se realizan en ambas metodologías, menos las auditorías, ya que el concepto de ser auditado y controlado por alguien externo, no es compatible con una metodología ágil pura.



Identificación de ítems de configuración

- ♡ identificación unívoca de cada ítem de configuración;
- ♡ convenciones y reglas de nombrado;
- ♡ definición de la Estructura del Repositorio;
- ♡ ubicación dentro de la estructura del repositorio.
- ♡ esto implica una identificación unívoca para cada ítem, donde en el equipo se definirán políticas y reglas de nombrado y versionado para todos ellos. También, se debe definir la estructura del repositorio, y la ubicación de los ítems de configuración dentro de esa estructura. Además, implica una definición cuidadosa de los componentes de la línea base.

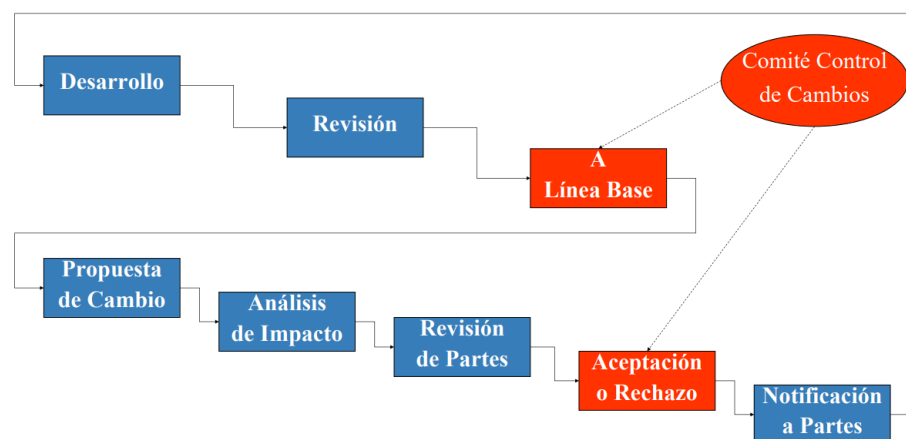
♥ Ítems de Configuración para un proyecto de desarrollo de software:

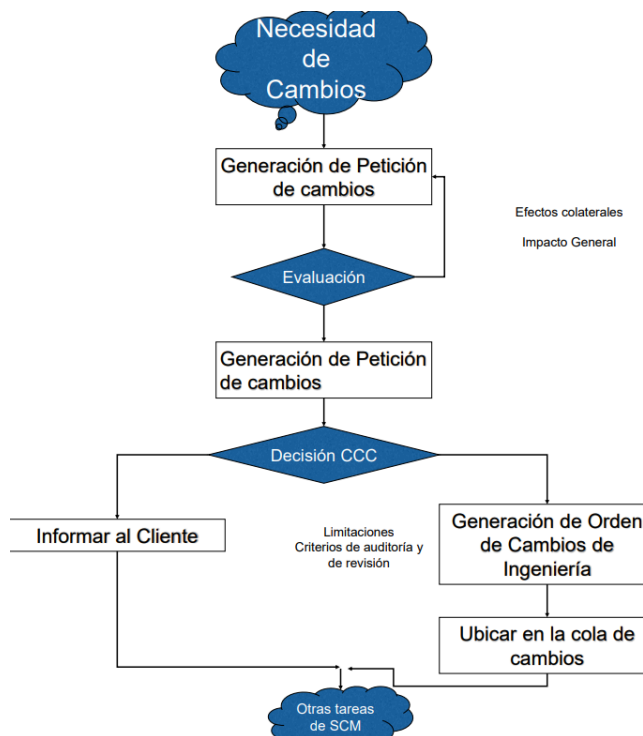


- Ítems de producto: tienen el ciclo de vida más largo, y se mantienen mientras el producto exista. Ejemplo: documento de arquitectura, casos de uso, código, manual de usuario y de parametrización, casos de prueba, una ERS, los casos de prueba, la base de datos.
- Ítems de proyecto: el plan de proyecto, el listado de defectos encontrados, tienen un ciclo de vida de proyecto. Un plan de iteración, un Burn-Down Chart, se mantiene durante una iteración. La duración impacta también en el esquema de nombrado, plan de proyecto. Ejemplo: Los ítems de configuración a nivel de proyecto, el plan de proyecto y el cronograma.
- Ítems de iteración: Conocer el ciclo de vida de un ítem permite establecer la nomenclatura de este. Ejemplo: planes de iteración, cronogramas de iteración, reporte de defectos.

Control de cambios

- ♥ Una vez definida la línea base, no es posible cambiarla sin antes pasar por un proceso formal de control de cambios. Es decir que el control se hace sobre los ítems de configuración que pertenecen a la línea base, ya que todos los trabajadores del software tienen a dichos ítems como referencia.
- ♥ Tiene su origen en un Requerimiento de Cambio a uno o varios ítems de configuración que se encuentran en una línea base.
- ♥ Es un Procedimiento formal que involucra diferentes actores y una evaluación del impacto del cambio





♥

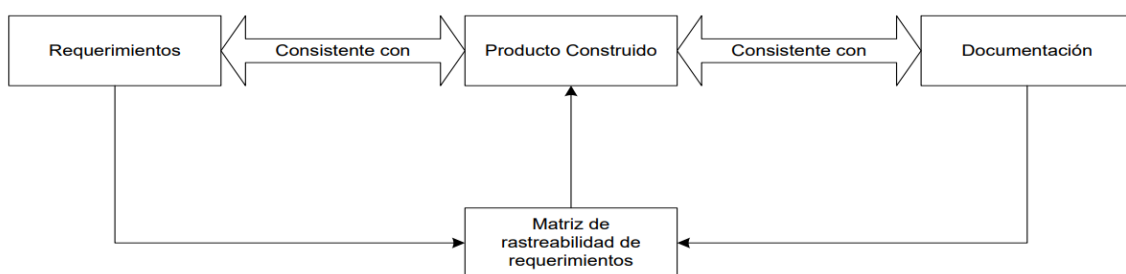
♥ Comité de control del cambios

- Está formado por representantes de todas las áreas involucradas en el desarrollo:
 - Análisis, Diseño
 - Implementación
 - Testing
 - Otros interesados

Auditorías de Configuración de Software

Auditoría Funcional de Configuración

Auditoría Física de Configuración



♥

Son controles autorizados a realizar por el equipo de desarrollo en un momento determinado, donde un auditor externo al equipo (independiente y objetivo) analiza las líneas base, las cuales permiten “congelar” y analizar en un momento determinado cuál es el estado del software, y si se están cumpliendo todas las pautas que plantea esta disciplina de SCM. En metodologías ágiles, esta es la única actividad de la disciplina SCM que no se soporta por la filosofía

♥

Auditoría física de configuración (PCA) Asegura que lo que está indicado para cada ICS en la línea base o en la actualización se ha alcanzado realmente.

♥

Auditoría funcional de configuración (FCA) Evaluación independiente de los productos de software, controlando que la funcionalidad y performance reales de

cada ítem de configuración sean consistentes con la especificación de requerimientos

- ♥ Auditoría de Gestión de Configuración y V&V
 - Sirve a dos procesos básicos: la validación y la verificación
 - Validación: el problema es resuelto de manera apropiada que el usuario obtenga el producto correcto.
 - Verificación: asegura que un producto cumple con los objetivos preestablecidos, definidos en la documentación de líneas base (línea base). Todas las funciones son llevadas a cabo con éxito y los test cases tengan status “ok” o bien consten como “problemas reportados” en la nota de release

Informe de estado

- ♥ provee un mecanismo para mantener un registro de cómo evoluciona el sistema, y dónde está ubicado el software, comparado con lo que está publicado en la línea base. Esto permite mantener a todo el equipo informado sobre la última línea base, y que se trabaje en base a su última versión, y no sobre una versión obsoleta.
- ♥ Estos reportes incluyen todos los cambios que han sido realizados a las líneas base durante el ciclo de vida del software. Normalmente esto consta de grandes unidades de datos, por lo que se utilizan herramientas automatizadas por una computadora. El objetivo principal es asegurarse que la información de los cambios llegue a todos los involucrados. El reporte más conocido es el de inventario, el cual provee una copia del contenido del repositorio con la estructura de directorios.
- ♥ Registro e Informe de Estado
 - Se ocupa de mantener los registros de la evolución del sistema.
 - Maneja mucha información y salidas por lo que se suele implementar dentro de procesos automáticos.
 - Incluye reportes de rastreabilidad de todos los cambios realizados a las líneas base durante el ciclo de vida.
- ♥ Algunas preguntas que podría responder
 - ¿Cuál es el estado del ítem?
 - ¿Un requerimiento de cambio ha sido aprobado o rechazado por el CCB?
 - ¿Qué versión de ítem implementa un requerimiento de cambio aprobado (saber cuál es el componente que contiene la mejora)?
 - ¿Cuál es la diferencia entre una versión y otra dada?

Plan de Gestión de Configuración

El plan de gestión de software también se planifica, y debería incluir:

- Reglas de nombrado de los CI
- Herramientas a utilizar para SCM
- Roles e integrantes del Comité
- Procedimiento formal de cambios
- Plantillas de formularios
- Procesos de Auditoría

e)Gestión de Configuración de Software en ambientes Ágiles

En las metodologías ágiles la gestión de configuración cambia el enfoque, ya que la misma es de utilidad para los miembros del equipo de desarrollo y no viceversa.

En orden con la caracterización de los equipos ágiles, decimos que la gestión de configuración posibilita el seguimiento y la coordinación del desarrollo en lugar de controlar a los desarrolladores.

- Sirve a los practicantes (equipo de desarrollo) y no viceversa.
- Hace seguimiento y coordina el desarrollo en lugar de controlar a los desarrolladores.
- Responde a los cambios en lugar de tratar de evitarlos.
- Esforzarse por ser transparente y "sin fricción", automatizando tanto como sea posible.
- Coordinación y automatización frecuente y rápida.
- Eliminar el desperdicio - no agregar nada más que valor.
- Documentación Lean y Trazabilidad.
- Feedback continuo y visible sobre calidad, estabilidad e integridad
- Tips:
 - ◆ Es responsabilidad de todo el equipo.
 - ◆ Automatizar lo más posible.
 - ◆ Educar al equipo.
 - ◆ Tareas de SCM embebidas en las demás tareas requeridas para alcanzar el objetivo del Sprint.

3)Requerimientos Ágiles

a)Filosofía Ágil y Manifiesto Ágil

Manifiesto ágil

Es un documento redactado en 2001 por expertos de la Ingeniería en Software, que promueve la filosofía del desarrollo ágil del software.

Valores Ágiles:

- Individuos e interacciones sobre procesos y herramientas: es preferible un equipo de personas motivadas con un buen funcionamiento y conocimiento, donde la comunicación sea fluida, pero con herramientas pobres para su trabajo, que tener un equipo de individuos desmotivados o poco funcional con las mejores herramientas y procesos. Así lo expresa el manifiesto, debido a que el desarrollo de software es una actividad humano-intensiva, y los procesos ágiles intentan capitalizar las fortalezas de cada individuo (como así también paliar las debilidades individuales), en lugar de intentar homogeneizar un equipo, para así lograr una armonía que permita fortalecer la unión y motivación dentro del equipo. Los procesos y herramientas deben ser un complemento para mejorar la eficiencia y éstos se deben adaptar al equipo y no al revés.

- software funcionando sobre documentación extensiva: priorizar el software asegura que se tengan versiones estables, incrementales y mejoradas del software al finalizar cada iteración. Esto permite que a medida que el software crezca en cuanto a funcionalidades, tras cada iteración puede ir mostrándose a los usuarios finales y obtener una retroalimentación de ellos, para que el equipo de desarrollo se asegure de estar trabajando siempre en aquellas funcionalidades de mayor valor para el cliente y aquellas que satisfagan sus expectativas. No plantea “no generar documentación”.
- Colaborar con el cliente sobre negociación contractual: esto permitirá no sólo tener una mejor noción de aquellos requerimientos de negocio que necesita el cliente, sino también que el equipo ágil se integre a todas las partes interesadas en el software, para que en conjunto puedan trabajar y lograr los mismos objetivos. Esto también reduce la dependencia de documentación extensiva ya que cada miembro del equipo participa en la toma de decisiones que solían requerir comunicación escrita.
- responder al cambio sobre seguir un plan: esto se plantea debido a que es imposible que los usuarios sepan con detalle y certeza todas las funcionalidades que necesitan, y a su vez, con el paso del tiempo surgirán nuevas necesidades, las cuales en un primer momento no se habían detectado. Esto está ligado al entorno cambiante en el que se desempeñan los clientes que demandan un software, por lo que un equipo ágil está abierto a los cambios, para poder responder ante ellos de una manera rápida y eficiente entregando el mayor valor posible para el cliente y los usuarios, en lugar de apegarse y seguir un plan definido confeccionado en etapas tempranas de requerimientos.

Principios del manifiesto ágil

1. Nuestra mayor prioridad es satisfacer al cliente
2. aceptar que los requisitos cambien
3. Entregar software funcional frecuentemente
4. los responsables de negocios, diseñadores y desarrolladores debe trabajar juntos día a día durante el proyecto
5. Desarrollamos proyectos en torno a individuos motivados
6. el método más eficiente de comunicar información es conversaciones cara a cara
7. el software funcionando es la principal medida de éxito
8. los procesos ágiles promueven el desarrollo sostenible
9. la atención continua a la excelencia técnica y al buen diseño mejor la agilidad
10. la simplificación es esencial
11. las mejores arquitecturas, requisitos y diseños emergen de equipos auto-organizados
12. a intervalos regulares el equipo reflexiona sobre como ser mas efectivo y de acuerdo a esto ajustan su comportamiento

Pilares del empirismo

Transparencia: la transparencia nos permite crecer como equipos y transformar el conocimiento que cada miembro del equipo tiene implícito en conocimiento explícito, es decir, que pase a formar parte del conocimiento de todo el equipo. Esto ayuda a que los procesos empíricos puedan fluir y generar esa experiencia necesaria para que el equipo evolucione.

Inspección: La inspección puede realizarse para el producto, procesos, aspectos de personas, prácticas y mejoras continuas. Se realiza por todo el equipo y permite detectar

varianzas o problemas potencialmente indeseables, se debe inspeccionar el progreso hacia los objetivos acordados.

Adaptación: Se trata de la mejora continua, es la capacidad de adaptarse en función de los resultados de la inspección. Nos permite hacer ajustes en aspectos del proceso que se desvían fuera de los límites o si el producto es inaceptable.

b)Ágil

NO es una metodología o proceso

Ágil es una ideología con un conjunto definido de principios que guían el desarrollo del producto

Valores de los equipos ágiles:

- Planificación continua, multi-nivel
- Facultados, auto-organizados, equipos completos
- Entregas frecuentes, iterativas y priorizadas
- Prácticas de ingeniería disciplinadas
- Integración continua
- Testing Concurrente

Desarrollo ágil de Software (Agile)

El desarrollo ágil adopta el ciclo de vida iterativo-incremental, donde el concepto del empirismo subyace al proceso, el software no se desarrolla como una sola unidad, sino como una serie de incrementos y cada uno de ellos incluye una nueva funcionalidad del sistema. Esta es una de las diferencias con el PUD o el RUP, que, si bien utilizan un modelo de proceso iterativo - incremental, son procesos definidos.

Un compromiso útil entre nada de proceso y demasiado proceso (Fowler, 2001)

¿Pero qué significa Ágil?

- Balance entre ningún proceso y demasiado proceso. La diferencia inmediata es la exigencia de una menor cantidad de documentación, sin embargo no es eso lo más importante:
 - Los métodos ágiles son adaptables en lugar de predictivos.
 - Los métodos ágiles son orientados a la gente en lugar de orientados al proceso.
- Frameworks ágiles:
 - Scrum
 - XP
 - ATDD
 - Crystal
 - Shape Up

c)Requerimientos Ágiles

El fondo de la cuestión está en los principios y valores explicitados en el manifiesto ágil, fundamentalmente la entrega frecuente y temprana de software valioso y la posibilidad de recibir cambios de requerimientos aun en etapas finales. Esto último implica una

predisposición, una actitud del equipo frente a la posibilidad de cambios. La base de los requerimientos en ambientes ágiles tiene que ver con la posibilidad de comenzar a construir el producto cuando el producto no está completamente definido

- Usar el “valor” para construir el producto correcto: Lo único importante es dejar contento al cliente con software funcionando que le sirva, por lo que, el foco de este enfoque de gestión ágil apunta a construir valor de negocio. Es decir, el software que creamos es para ayudar a alguien más a cumplir con sus objetivos y trabajo, por eso decimos que debemos enfocarnos en construir el producto correcto que le dé valor al negocio. Hay que construir un producto que le haga la vida más fácil a las personas que lo utilizan.
- Usar historias y modelos para mostrar que construir :la parte de desarrollo de requerimientos está muy enfocada a la idea de construir junto con el cliente, respetando el principio ágil de “técnicos y no técnicos trabajando juntos”. Se busca trabajar con un referente que esté del lado del negocio, que tenga el conocimiento del producto y que trabaje junto con el equipo
- Determinar que es “sólo lo suficiente”: hace referencia a que los requerimientos deben ser encontrados de a poco, no buscar todos los requerimientos desde un inicio y quedarnos solo con eso. Este pilar postula que se tiene que empezar con lo mínimo necesario para generar realimentación y alimentar el empirismo y a partir de allí, se avanza continuamente.

BRUF – Big Requirements Up Front

Los productos “Exitosos” también tienen un desperdicio significativo (BRUF – Big Requirements Up Front)

La imagen adjuntada expresa la frecuencia con la que los usuarios utilizan determinadas funcionalidades de un software. A lo que se apunta con este concepto de BRUF es a notar que, en una primera instancia de un proyecto de software, desarrollar sólo unas pocas funcionalidades de gran valor e importancia para el cliente, puede cubrir gran parte de sus necesidades, a medida que se desarrolla el resto de los requerimientos. Además, es prácticamente imposible conocer al 100% todas las necesidades de los usuarios, ya que ni ellos saben concretamente qué necesitan, ni cómo explicar de forma abarcadora y completa sus necesidades.

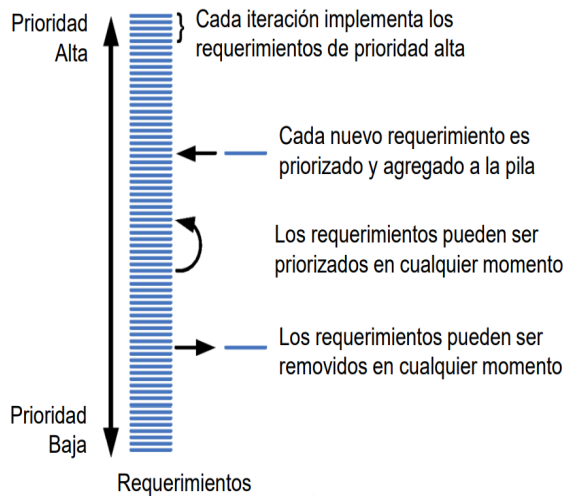


Gestión Ágil de Requerimientos de Software

Recordemos que la ingeniería de requerimientos tiene 2 procesos: por un lado, el desarrollo de requerimientos y por otro lado la gestión de los requerimientos. La gestión está más vinculada con mantener los requerimientos íntegros, bien identificados, bien trazados y que si cambian los requerimientos se actualicen todos los lugares donde se tengan que actualizar esa definición de requerimientos. Esto ocurre todo el tiempo mientras estamos trabajando con un producto de software. La parte de desarrollo de requerimientos, que es la parte que tiene que ver con identificar, especificar y validar los requerimientos.

Product Backlog

Los requisitos cambiantes son una ventaja competitiva si puede actuar sobre ellos



Product Backlog: La gestión ágil busca contrarrestar la situación planteada con anterioridad donde gran parte del software no se termina utilizando. El Product Backlog es una lista priorizada de características y requerimientos del software. Es el Product Owner quien se encarga de definir esta lista ya que es él quien tiene en claro cuáles son sus necesidades y requerimientos principales. El Product Owner se va a encargar de decidir qué requerimientos necesita primero según el valor que éstos tengan para el negocio.

Requerimientos Just in Time: La gestión ágil de requerimientos sostiene los

requerimientos justo a tiempo, asociados a los principios de eliminar el desperdicio y diferir compromisos, dentro de la filosofía Lean. Se empieza con una visión del producto y con algunos requerimientos identificados. A partir de esta base, se comienza a construir el producto y se incorpora al cliente para obtener retroalimentación. El Product Backlog nunca está al 100%

Fuente de requerimientos

El cara-a-cara permite que fluya información vocal, subvocal, gestual con realimentación rápida.



-los requerimientos de sw se obtienen conversando cara a cara con el cliente
-En ambientes tradicionales los req se encuentran en el documento de especificación de req (ERS) y el cliente lo lee para validarlo
-Los agilistas no escriben una ERS, pero esto se compensa con la disponibilidad del cliente para resolver dudas, reduciendo la cantidad de documentación formal

-El desarrollo ágil no implica la no documentación o la no especificación de req, sino que prioriza documentar y especificar aquellos que sean de valor para el cliente, antes que hacer toda una ERS errónea.

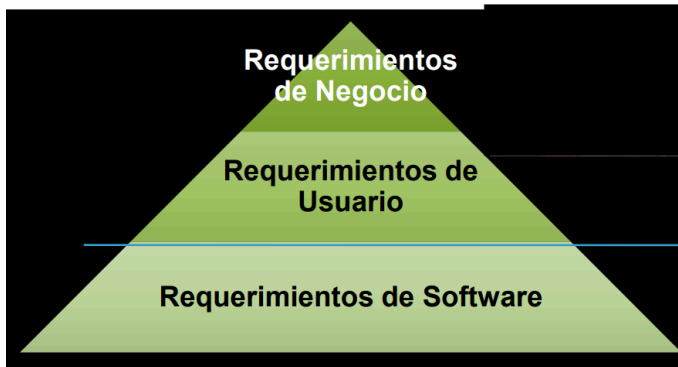
Valor "El valor lo asociamos a la utilidad, beneficio o satisfacción que le ofreces a los usuarios finales por cada funcionalidad completa que le entregas"

"Valor es la obtención de beneficio tangible o intangible"

Tipos de requerimientos

- **Requerimiento de Negocio:** Disminuir un X% de tiempo invertido en procesos manuales relacionados con atención al cliente.
- **Requerimiento de Usuario:** Realizar consultas en línea del estado de cuenta de los clientes (Las User Stories son requerimientos de usuario alineados a un requerimiento de negocio)

- Requerimiento Funcional: Generar reporte de saldos de cuenta. Recibir notificaciones por
- Requerimiento No funcional: Formato del reporte PDF. Cumplir niveles de seguridad para credenciales de usuarios según la ley bancaria 9999XX
- Requerimiento de Implementación: Servidores en la nube



Dominio del problema (De negocio de usuario)

Dominio de la solución (Software)

Tradicional vs. ágil

En el enfoque ágil, se trata de aceptar la realidad en la cual el alcance (requerimientos) del producto puede cambiar a lo largo del desarrollo, y no es posible definirlos al comienzo de este. Entonces, el enfoque trata de dejar el tiempo y los recursos fijos, teniendo en cuenta “cuánto software” le puedo entregar en tanto tiempo y con tal equipo de desarrollo. Por otro lado, en el enfoque tradicional, el alcance se fija y en función de eso derivan los recursos (costo) y el tiempo. El cliente desea controlar el alcance, ya que este comprende lo que él realmente necesita. Luego el cliente pregunta por el tiempo y el costo, los cuales son derivados por el equipo. Por esta razón es tan costosa incorporar cambios de requerimientos, ya que se estaría modificando el alcance y como consecuencia los recursos y el tiempo.



Se debe asumir que:

- Los cambios son la única constante.
- Stakeholders: no son todos los que están. Debemos saber quiénes son los involucrados y los involucrados son todos los que tiene algo que decir respecto del producto. El PO es el representante de todas estas personas y el que se comunica con todas ellas para luego ser él mismo el que se comunica con el equipo que hace el software.
- Siempre se cumple eso de que: “El usuario dice lo que quiere cuando recibe lo que pidió”.

- No hay técnicas ni herramientas que sirvan para todos los casos.
- Lo importante no es entregar una salida, un requerimiento, lo importante es entregar, un resultado, una solución de “valor”.

Principios ágiles relacionados con requerimientos ágiles



1- LA PRIORIDAD ES SATISFACER AL CLIENTE A TRAVÉS DE RELEASES TEMPRANOS Y FRECUENTES (2 SEMANAS A UN MES)



2 -RECIBIR CAMBIOS DE REQUERIMIENTOS, AUN EN ETAPAS FINALES



4 - TÉCNICOS Y NO TÉCNICOS TRABAJANDO JUNTOS TODO EL PROYECTO



6 - EL MEDIO DE COMUNICACIÓN POR EXCELENCIA ES CARA A CARA



11 - LAS MEJORES ARQUITECTURAS, DISEÑOS Y REQUERIMIENTOS EMERGEN DE EQUIPOS AUTOORGANIZADOS

d)User Stories

Son una técnica para trabajar requerimientos de usuario en ambientes ágiles. La US contiene una descripción corta de una funcionalidad que se espera del producto valuada por un usuario del sistema.

Internamente, las User Stories se consideran verticales dentro del producto, ya que pueden incluir aspectos desde diseños de interfaces hasta diseños de tablas en la base de datos.

Son multipropósito, las historias son:

- Una necesidad del usuario
- Una descripción del producto
- Un ítem de planificación
- Token para una conversación
- Mecanismo para diferir una conversación

Partes de la US

-Tarjeta: es una descripción de la historia, utilizada para planificar y como recordatorio.

Tiene una sintaxis particular que es la siguiente:

Como <nombre del rol> , yo puedo <actividad> de forma tal que <valor de negocio que recibo>

<<Frase Verbal>>

Como <<nombre del rol>> yo puedo <<actividad>> de forma tal que <<valor de negocio>>

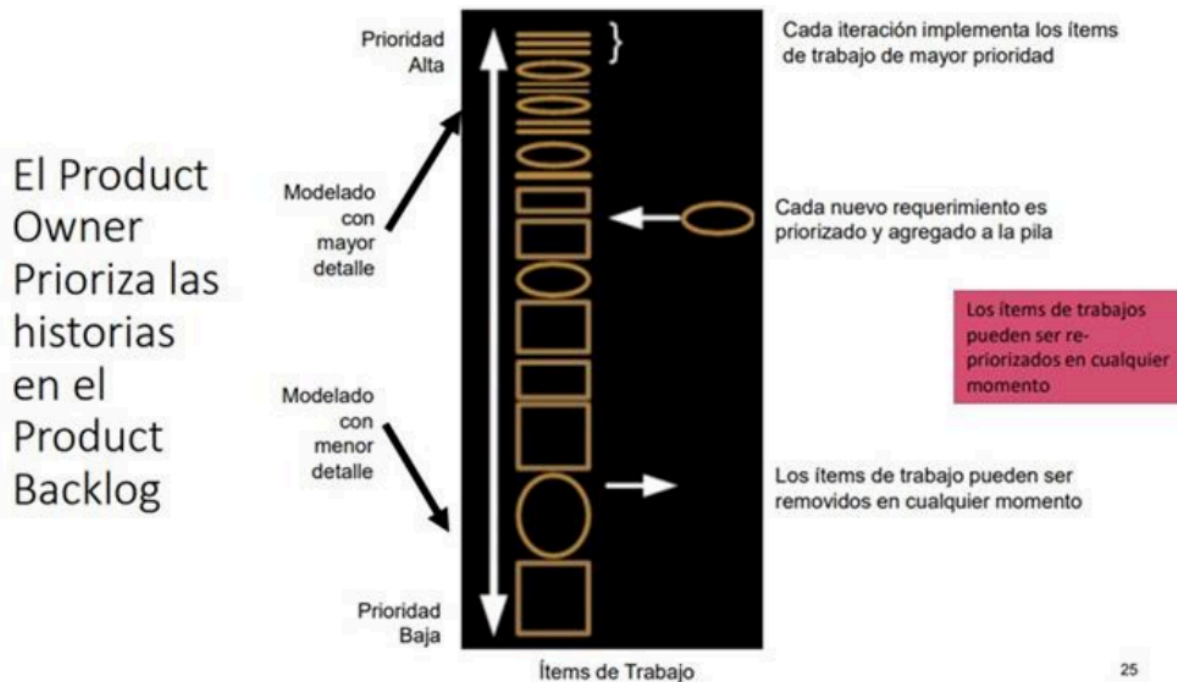
- Nombre del rol: representa quien está realizando la acción o quien recibe el valor de la actividad.
- Actividad: representa la acción que realizará el sistema.
- Valor de negocio: representa el por qué es necesaria la actividad.
- Frase verbal: no es obligatoria, es una forma corta de referenciar la US. Representa la funcionalidad que se expresa en la User Story

- Criterios de aceptación: son diferentes criterios sobre la actividad que se necesita implementar
 - Definen límites para una user story (US)
 - Ayudan a que los PO respondan lo que necesitan para que la US provea valor (requerimientos funcionales mínimos)
 - Ayudan a que el equipo tenga una visión compartida de la US
 - Ayudan a desarrolladores y testers a derivar las pruebas.
 - Ayudan a los desarrolladores a saber cuando parar de agregar funcionalidad en una US
 - Cuáles son los criterios de aceptación buenos?:
 - Definen una intención, no una solución
 - Son independientes de la implementación
 - Relativamente de alto nivel, no es necesario que se escriba cada detalle
 - ¿Dónde van los detalles entonces? Estos detalles que son el resultado de las conversaciones con el PO y el equipo puede capturarlos en dos lugares:
 - Documentación interna de los equipos
 - Pruebas de aceptación automatizadas
- Pruebas de aceptación: acompañan a la tarjeta en la parte trasera, y es donde se capturan todos los detalles (que se manifiestan en la conversación) de la User, permitiendo determinar cuándo una historia está completa. Esto servirá a los desarrolladores para probar si la implementación ha sido realizada de forma correcta y completa.
 - Expresan detalles resultantes de la conversación y complementan la User Story
 - Proceso de dos pasos: 1. Identificarlas al dorso de la US. 2. Diseñar las pruebas completas

-Conversación: Es la parte más importante de la US, ya que explicita la comunicación entre el Product Owner y el Equipo de desarrollo que permite compensar la falta de especificación y detalle. El acuerdo por sobre la negociación del contrato y el principio de técnicos y no técnicos trabajando juntos durante todo el proyecto están asociadas a esta parte de la US. El Product Owner es parte del equipo, por lo cual se espera responsabilidad, compromiso y confianza de su parte para hacerse cargo y llegar a un acuerdo común con el equipo sobre lo definido acerca de la funcionalidad

-Confirmación: Definición de un acuerdo entre el Product Owner y el Equipo para demostrar que la característica del producto realmente se implementó. Conforma a la definición de las pruebas de aceptación. El PO decide si las pruebas de aceptación son válidas. Pruebas que se usan para determinar cuándo una US está completa.

Product Backlog y las User Stories



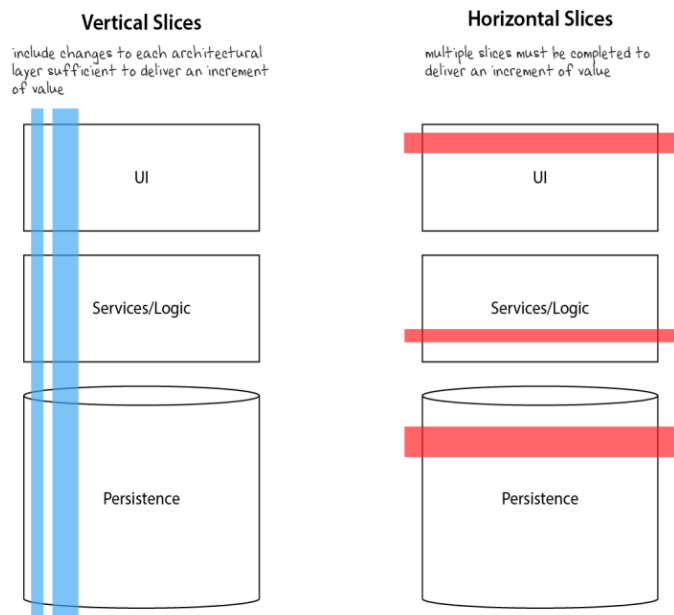
25

Es una pila de PBIs (Product Backlog Items) con sus correspondientes estimaciones de tamaño, en un formato determinado. En la parte más alta de la pila, se ubican las U.S. de mayor prioridad (que son aquellas U.S. con granularidad fina), y las de menor prioridad (normalmente aquellas User de granularidad gruesa, sobre las cuales no se tiene mucho detalle ya que no se consideran indispensables) al fondo. En cualquier momento se puede cambiar esa priorización, según cómo lo indique el Product Owner, quien es el encargado de realizar el proceso de priorización.

Para quitar una U.S. del Product Backlog, se planifica una iteración y se decide, según la velocidad de trabajo del equipo [Story Points / iteración], cuántas U.S. de la pila (las de mayor prioridad que están arriba) se quitarán para implementarlas en la próxima iteración.

Porciones verticales

Las US son porciones verticales, es decir, para poder agregar valor al cliente debemos tener un poco de interfaz, un poco de lógica de negocio y otro poco de base de datos. Debemos diseñar el pedacito de cada cosa que nos hace falta para que una determinada característica funcione.



Story 1	Story 2	
GUI		
Business Logic		
Database		

Modelado de roles

Se utilizan las tarjetas de rol de usuario, aquí se anotan las características en general que va a tener el usuario. El tema crítico de la funcionalidad está relacionado con quien va a utilizar dicha funcionalidad. Por esta razón existe una tendencia por ocuparse de quien va a usar el sistema, especificando los roles de usuario donde definimos las características del usuario respecto al producto y en relación con su perfil genérico.

Técnicas adicionales

- **Personas:** esta técnica busca ser más específica acerca de la descripción de la persona concreta que va a utilizar el sistema. Este análisis es exhaustivo y por lo tanto complejo. Si los roles son clases, las personas serían los objetos.
- **Personajes Extremos:** que personas en términos extremos podrían llegar a utilizar el software
- **Usuarios Representantes (Proxies):** es un representante de todos los usuarios. Debe tener capacidad de tomar decisiones desde el negocio.

DoR (Definition of Ready) – Definición de Listo Es una medida de calidad que determina si la User Story está en condiciones de entrar a una iteración de desarrollo. Para que la US pueda ser implementada, mínimamente debe satisfacer el INVEST Model. Entre el equipo de desarrollo y el PO pueden definir características o condiciones extras al INVEST para que una US se considere lista para entrar a la iteración de desarrollo.

DoD (Definition of Done) – Definición de Hecho Determina si la US está terminada. Son los criterios que establecen cuándo una User Story ha sido implementada de forma correcta y completa, de forma que puede ser mostrada al Product Owner, para que éste decida si se pasa a producción o no. En el momento que una U.S. cumple con todo lo pactado en esta Definición de Hecho, se considera que esa US está lista para ser desplegada y aportar valor al cliente.

Esto permite crear transparencia, debido a que proporciona a todo el equipo una comprensión exhaustiva de qué trabajo se realizó, para lograr ese incremento.

En la práctica consta de un Checklist de condiciones que deben cumplirse.

INVEST Model Es un modelo de calidad que nos ayuda a verificar si la User cuenta con las características de la calidad mínima para satisfacer la definición de Ready y poder ingresar en una iteración de desarrollo.

- **Independent** – calendarizables e implementables en cualquier orden. Esto implica que no existe una dependencia con otra User para desarrollarla o para mostrarla al Product Owner
- **Negotiable** – el “qué” no el “cómo”. El cliente debe expresar lo que necesita, es decir que está centrada en el “qué” no el “cómo”. El cómo es decisión del equipo y debe ser negociable con el Product Owner. La US debe estar escrita en términos de qué necesita el usuario y no de cómo lo vamos a implementar.
- **Valuable** – debe tener valor para el cliente
- **Estimable** – para ayudar al cliente a armar un ranking basado en costos. La US debe contener la cantidad de información suficiente para estimar el tamaño, la complejidad y el esfuerzo necesario para realizarlo.
- **Small** – deben ser “consumidas” en una iteración

- Testable – la US debe cumplir con las pruebas de aceptación para poder probar si la misma fue o no implementada (se muestra que está hecha y demostrable).

Niveles de abstracción



- User Story: cumple con el criterio de Listo (DoR) para ingresar a una iteración.
- Épica: es una User muy grande. El tamaño es relativo a si entra en una iteración o no, ya que las User se empiezan y se terminan en la iteración. Si la User no entra en la iteración es considerado una épica. Tienen la misma sintaxis que la User. Una épica se suele identificar también cuando no cumple con el modelo de calidad de INVEST. La épica debe ser dividida en US más pequeñas. Sirven para aquellos requerimientos que no son prioridades en las primeras iteraciones, y basta con tener sólo una aproximación general de las funcionalidades requeridas. Llegado el momento de su implementación, se deberán dividir en historias de usuario más pequeñas, para poder planificar su implementación en sucesivas iteraciones.
- Temas: son un conjunto de User Stories agrupadas, debido a que conceptualmente están relacionadas. Se las agrupa para tratarlas como una entidad simple a la hora de estimar o planificar. Ambos (Épicas y Temas) permiten reducir los esfuerzos del equipo a la hora de estimar.

Spikes

Son un tipo especial de US ue se producen por la incertidumbre que la misma presenta, la cual imposibilita que pueda ser estimada y por lo tanto no cumple con la definición de listo, Una vez resuelta la incertidumbre, la Spike se convierte en una o más US.

Pueden utilizarse para:

- Inversión básica para familiarizar al equipo con una nueva tecnología o dominio.
- Analizar un comportamiento de una historia compleja y poder así dividirla en piezas manejables.
- Ganar confianza frente a riesgos tecnológicos, investigando o prototipando para ganar confianza.

- Frente a riesgos funcionales, donde no está claro como el sistema debe resolverla interacción con el usuario para alcanzar el beneficio esperado.

Tipos:

- Técnicas: Utilizadas para investigar enfoques técnicos en el dominio de la solución. está asociado a la implementación de la funcionalidad. Nueva tecnología. Cualquier situación en la que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo.
- Funcionales: Utilizadas cuando hay cierta incertidumbre respecto de cómo el usuario interactuará con el sistema. Usualmente son mejor evaluadas con prototipos para obtener realimentación de los usuarios o involucrados.

Lineamientos para spikes

- ★ Estimables, demostrables, y aceptables
- ★ La excepción, no la regla
 - Toda historia tiene incertidumbre y riesgos.
 - El objetivo del equipo es aprender a aceptar y resolver cierta incertidumbre en cada iteración.
 - Los spikes deben dejarse para incógnitas mas críticas y grandes.
 - Utilizar spikes como última opción.
- ★ Implementar la spike en una iteración separada de las historias resultantes
 - Salvo que el spike sea pequeño y sencillo y sea probable encontrar una solución rápida en cuyo caso, spike e historia pueden incluirse en la misma iteración.

Aclaraciones



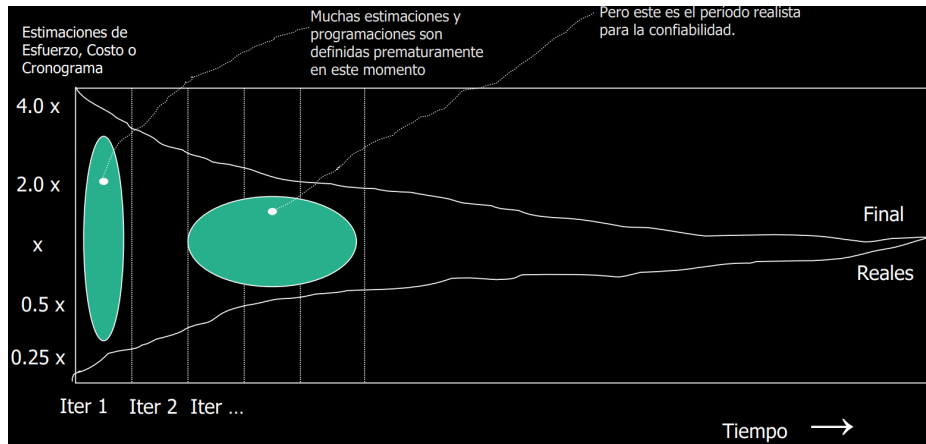
4) Estimaciones de software

Estimar es el proceso de obtener una aproximación sobre una medida con el objetivo de predecir la completitud y gestionar los riesgos en el contexto de un proyecto. Se relaciona con los objetivos del negocio, compromisos y control.

- Por definición una estimación no es precisa.
- A mayor diferencia entre lo estimado y lo planeado mayor riesgo

- Estimar no es planear y planear no es estimar.
- Las estimaciones no son compromisos.
- Las estimaciones son la base de los planes, pero los planes no tienen que ser lo mismo que lo estimado.

a) ¿Para qué estimamos?



Antes de que el proyecto comience, el líder del proyecto y el equipo de software deben estimar el trabajo que habrá de realizarse, los recursos que se requerirán y el tiempo que transcurrirá desde el principio hasta el final. Las estimaciones requieren:

- Experiencia.
- Acceso a buena información histórica (métricas).
- El valor para comprometerse con predicciones cuantitativas cuando la información cualitativa es todo lo que existe.

Errores en las estimaciones

1. Información imprecisa acerca del software a estimar o acerca de la capacidad para realizar el proyecto.
2. Demasiado caos en el proyecto (mal definido el proyecto).
3. Imprecisión generada por el proceso de estimación.
4. Una de las fuentes más común es omitir actividades necesarias para la estimación del proyecto tales como, requerimientos faltantes, licencias, reuniones, revisiones, etc.
5. Se olvidan del detalle que planear no es estimar y estimar no es planear, sino que las estimaciones son la base de los planes, pero los planes no tienen que ser lo mismo que lo estimado.
6. A mayor diferencia entre lo planeado y lo estimado, mayor el riesgo.
7. Las estimaciones no son compromisos.

b) Técnicas Fundamentales de estimación

Contar: Para contar podemos utilizar varios métodos basados en distintas cosas (se utiliza más de uno)

Basados en la experiencia

- **Datos históricos:** consiste en recolectar datos básicos de otros proyectos para ir generando una base de conocimientos para estimaciones, con esto se busca una alternativa en la cual el conocimiento de cada individuo se transfiere a la organización y no estamos atados a una única persona. Surge la problemática del guardado de los datos, extrapolarlos y compararlos
- **Juicio experto:** Es el enfoque de estimaciones más utilizado en la práctica. • Acerca del 75% de organizaciones de software usan principalmente "juicio de experto"
 - **Puro:** un experto estudia las especificaciones y realiza una estimación. Esta estimación se basa fundamentalmente en la experiencia del experto. (el problema es que, si el experto se va de la organización, esta pierde su capacidad de estimación). (otra es cómo definimos al experto y cómo validamos sus conocimientos)
 - Estructurando el juicio de experto: Tenga tareas una granularidad aceptable. • Use el método de "optimista, pesimista y habitual" y su formula = $(o + 4h + p)/6$ • Use un checklist y un criterio definido para asegurar cobertura.
 - **Wideband Delphi:** Un grupo de personas (expertos) son informadas y tratan de adivinar lo que costará el desarrollo tanto en esfuerzo, como en duración. Las estimaciones en grupo suelen ser mejores que las individuales. (un grupo de personas se reúne con el objetivo de converger a una estimación común, tanto en esfuerzo como en duración)
 - Proceso:
 - Se dan las especificaciones a un grupo de expertos.
 - Se les reúne para que discutan tanto el producto como la estimación.
 - Remiten sus estimaciones individuales al coordinador.
 - Cada estimador recibe información sobre su estimación, y las ajenas pero de forma anónima.
 - Se reúnen de nuevo para discutir las estimaciones.
 - Cada uno revisa su propia estimación y la envía al coordinador.
 - Se repite el proceso hasta que la estimación converge de forma razonable.
 - Este método es tomado como base para el Poker Planning.

Analogía Se comparan los factores a estimar de forma relativa con respecto a las estimaciones de otros proyectos similares. Es un método que tiene mucho error dado que el conocimiento en un proyecto no es extrapolable a otro. Es decir, consiste en tomar a un proyecto e ir comparando factores

- Tamaño: es menor o mayor al proyecto anterior.
- Complejidad: más complejo de lo usual.
- Usuarios: si hay más usuarios habrá más complicaciones, como la concurrencia o los datos a almacenar.

Otros: Basados exclusivamente en los recursos , Basados en la capacidad del cliente o exclusivamente en el mercado, Basados en los componentes del producto o en el proceso de desarrollo , Métodos algorítmicos

c)Actividades Omitidas

Una de las fuentes de error mas común en las estimaciones es omitir actividades necesarias para la estimación del proyecto.

- Requerimientos faltantes.
- Actividades de desarrollo faltantes (documentación técnica, participación en revisiones, creación de datos para el testing, mantenimiento de producto en previas versiones)
- Actividades generales. (días por enfermedad, licencias, cursos, reuniones de compañía).
- Uso de buffers

5)Estimaciones en ambientes Ágiles

a)Tips respecto a las estimaciones

- Si las estimaciones se utilizan como compromisos son muy peligrosas y perjudiciales para cualquier organización.
- Lo más beneficioso en las estimaciones es el “proceso de hacerlas”.
- La estimación podría servir como una gran respuesta temprana sobre si el trabajo planificado es factible o no.
- La estimación puede servir como una gran protección para el equipo.

Problemas de estimaciones tradicionales

- Estimar demasiado pronto: Las estimaciones tempranas son inherentemente erróneas. No podemos planificar para eliminar la incertidumbre; debemos iterar para descubrirla.
- Resistencia a las reestimaciones.
- Estimaciones que buscan precisión.
- Estimaciones hechas por gente distinta a la que hace el trabajo.
- Estimaciones que pretenden ser absolutas (inflexibles).

Anatomía de una buena estimación

- Expresada en rangos, no en puntos únicos. (Ej: '12 a 16 semanas').
- Incluye un porcentaje de probabilidad explícito. (Ej: 'Con un 75% de confianza').
- Totalmente separada del Objetivo de negocio. (No fue modificada para 'encajar' en una fecha deseada).
- Habilita decisiones de control. (Da visibilidad para negociar alcance, recursos o fechas reales).

b)Estimaciones en ambientes ágiles

Son presunciones numéricas asociadas a una probabilidad de que cierto evento ocurra. La filosofía ágil plantea que quien estima, es quien debe hacer el trabajo, con lo cual es el equipo de desarrollo quien realiza las estimaciones, de forma colectiva (menos el Product Owner, no participa en estimaciones). Al ser de esta manera, la estimación se ajusta mejor, debido a que se generan debates entre los miembros del equipo, lo cual puede abrir puntos de vista no detectados por otros, haciendo que ese proceso de estimar se vuelva lo más preciso posible (a diferencia de las metodologías tradicionales, donde se estima individualmente y se depende sólo de ese miembro).

- Las features/stories son estimadas usando una medida de tamaño relativo conocida como story points (SP)
- Las medidas relativas no son absolutas.
- Story Points no es una medida basada en tiempo

Estimación Relativa / ¿Qué se estima?

Lo que se estima en ambientes ágiles, es solo el tamaño de las US (o Features) usando como unidad de medida los Story Points, y teniendo en cuenta 3 aspectos de estas: incertidumbre, complejidad y esfuerzo. Como la complejidad de las US tiende a incrementarse exponencialmente, se utilizan comúnmente series numéricas de esta índole, siendo una de las más comunes, la serie de Fibonacci.

Al ser estimaciones de tamaños relativos, (es decir, se estima realizando comparaciones de los tamaños de las US) lo importante en este proceso no es el valor que se asigna a cada US en particular, sino más bien que las estimaciones sean consistentes en el conjunto. Es decir, para un equipo, una US de 8 puntos puede no significar lo mismo que para otro equipo, pero lo que importa es que en un mismo equipo, una US de 5 puntos, siempre sea más chica que una de 8.

Es importante tener en claro que no se estima tiempo, ya que la duración de los Sprints es fija (concepto de Timebox). Por esto, es que no se recomienda estimar el tamaño de las US en horas, para no confundir con el esfuerzo.

- Las personas no saben estimar en términos absolutos
- Somos buenos comparando cosas
- Comparar es generalmente más rápido.
- Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual
- Se emplea mejor el tiempo de análisis de las storys

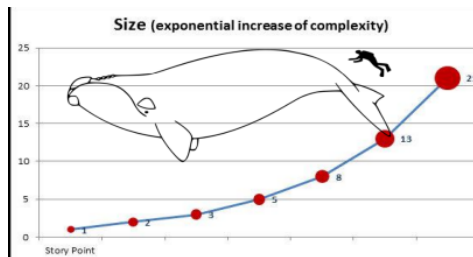
Los humanos somos pésimos estimando absolutos (horas), pero excelentes en comparaciones relativas ("esta tarea es el doble de compleja que aquella").

Usamos la escala de Fibonacci (1, 2, 3, 5, 8, 13) para reflejar que a mayor tamaño, mayor es la incertidumbre.

Story Point

- ☆ Es una unidad de medida específica (del equipo) de, complejidad, riesgo y esfuerzo, es lo que "el kilo" a la unidad de nuestro sistema de medición de peso
- ☆ Story point da idea del "peso" de cada story y decide cuan grande (compleja) es
- ☆ La complejidad de una feature/story tiende a incrementarse exponencialmente.
 - Complejidad: que tan compleja es implementar esa US. Relacionada a la cantidad de partes y relaciones entre las mismas posee la US.
 - Esfuerzo: es importante que estime la persona que va a hacer el trabajo, ya que el esfuerzo es distinto entre dos personas. Son las horas persona lineales que se requieren para terminar la US.
 - Incertidumbre: es cuánta información falta para ser más adecuados en la estimación. Existe una incertidumbre técnica que es más sobre si se sabe cómo llevar a cabo lo explicitado con respecto a la tecnología a implementar

y ese tipo de cosas. Y tenemos incertidumbre del negocio, si entendemos que es lo que el cliente pide y cómo llevarlo a cabo.



Tamaño vs. Esfuerzo

- El tamaño es una medida de la cantidad de trabajo necesaria para producir una feature/story.
 - El tamaño indica:
 - Cuán compleja es una feature/story
 - Cuánto trabajo es requerido para hacer o completar una feature/story
 - Y cuán grande es una feature/story
 - Existen diferentes escalas para estimar el tamaño y siempre son por comparación, Una vez definida la escala, no se cambia
- La complejidad es independiente de quien haga el proyecto, y el esfuerzo depende específicamente de la persona que lo va a llevar a cabo, de su capacitación, conlleva horas lineales (horas únicamente de trabajo, sin tener en cuenta almuerzo, baño, leer emails, etc. Para una jornada de trabajo de 8 horas, usualmente solo 5.5 o 6 son lineales).

Velocidad

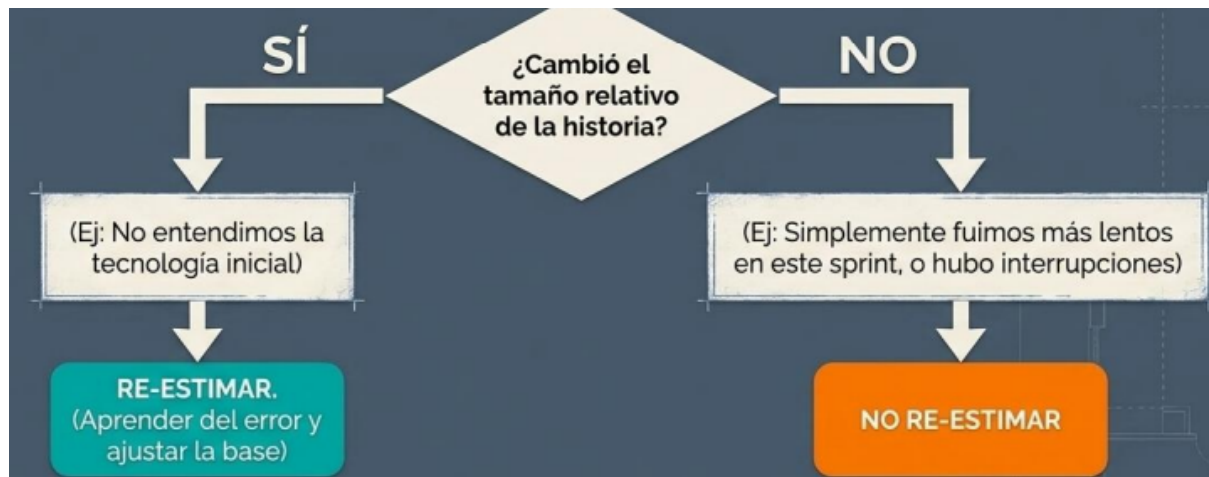
- ✿ Velocidad/ Velocity es una medida (métrica) del progreso de un equipo. Se calcula sumando el número de story points (asignados a cada user story) que el equipo completa durante la iteración.
- ✿ Se cuentan los story points de las Users Storys que están completas, no parcialmente completas.
- ✿ La Velocidad corrige los errores de estimación.

Si la US se completó parcialmente, no se cuentan sus Story Points en esa iteración (no existe el concepto de casi-completo, es una gestión binaria, es decir, se completó o no se completó la US). Esto genera que la velocidad del equipo baje en esa iteración, pero en la siguiente, aumentará, debido a que en unos pocos días probablemente se complete la US pendiente, y se pueda continuar con otras US planificadas en la iteración.

Esto implica que el equipo no se enfoque en la velocidad de cada iteración, sino más bien, en el promedio de velocidades a lo largo de sucesivas iteraciones. Esto brindará una mayor certeza acerca de la medida de progreso de trabajo del equipo.

Reestimación

Re-estimar una tarea solo porque tomó más tiempo del esperado arruina la línea base matemática del proyecto. Dejé que la Velocidad absorba el impacto.



Estimar nuevamente una US es aconsejable sólo cuando la opinión acerca del tamaño relativo de una US ha cambiado, debido a que en el proceso de estimación se han detectado otras U.S. con tamaños más grandes o pequeños. (esto tiene que ver con mantener la consistencia de tamaños a la hora de estimar).

Negociar el Compromiso, nunca la Estimación.

Equipo Técnico : Dueños absolutos de la Estimación.

- Una estimación es un cálculo basado en datos; negociarla es tan inútil como negociar la temperatura del sol.

Negocio: Dueños absolutos del Objetivo.

- Lo que Sí se negocia es el Compromiso: ajustar el alcance (scope), mover la fecha, o aceptar más riesgo de negocio.

Hay un COMPROMISO entre ambos

c)Poker estimation (método de estimación)

Popular entre los Agile practitioners, publicado por Mike Cohn. Combina opinión de experto, analogía y desegregación.

Asume que las personas que desarrollan el producto deben ser aquellas que estimen. En otras palabras, el equipo estima su propio trabajo y los miembros opinan sobre lo mismo, donde se mezcla la experiencia y el conocimiento de cada uno con la posibilidad de compartir con el resto del grupo.

Participantes en “planning poker” son desarrolladores → “Las personas más competentes en resolver una tarea deben ser quienes las estiman”

Escala

Dado que la complejidad en el software incrementa de forma exponencial, se decide adoptar la serie de Fibonacci como escala para la determinación de los puntos de historia.

- Valor 1 o 1/2: son los valores más bajos y se aplican a una funcionalidad pequeña. 2.
- Valor 2- 3: funcionalidad pequeña a mediana.
- Valor 5: funcionalidad media.
- Valor 8: funcionalidad grande, donde nos preguntamos si es posible dividir.
- Valor 13 o más: necesariamente hay que dividir ya que no ingresa en el sprint.
- Valor 20:Cuál es la razón de negocio que justifica semejante story y más fuerte aún, por qué no se puede dividir?.

- Valor 40: no hay forma de hacer esto en un sprint.
- Valor 100: confirmación de que está algo muy mal. Mejor ni arrancar.

Herramienta colaborativa

Combina la opinión de expertos, evita el sesgo de anclaje y fuerza las conversaciones técnicas más valiosas precisamente cuando las estimaciones divergen.

Procedimiento y prerequisites

- Prerequisites:
 - Lista de features/stories a ser estimadas
 - Cada estimador tiene un mazo de cartas.
- Pasos:
 1. Determine la base story (la canónica) que será usada para comprar con las otras stories. Digamos, Story Z.
 - 1.1 La story a ser estimada se lee a todo el equipo.
 - 1.2 Los estimadores discuten la story, haciendo preguntas al product owner (las que se necesiten).
 - 1.3 Cada estimador selecciona una carta y pone la carta boca abajo en la mesa.
 - 1.4 Cuando todos pusieron las cartas, las mismas se exponen al mismo tiempo.
 - 1.5 Si todos los estimadores selecciona el mismo valor, ese es el estimado. Sino, los estimadores discuten sus resultados, poniendo especial atención en los más altos y los más bajos. Después de la charla, GOTO to 1.3.
 2. Se toma la próxima story, se discute con el product owner.
 3. Cada estimador asigna a la story un valor por comparación contra la base story. "Cuan grande/pequeña, compleja, riesgosa es esta story comparada con Story Z?". GOTO to 1.3

Para realizar una estimación se debe tener en cuenta el criterio de Done y no solo la tarea de programación, es decir todo lo que hay que hacer para terminar una US y poder lidiar con el problema de las actividades omitidas.

El tiempo para realizar las estimaciones, como cualquier otra actividad en los ambientes ágiles, es time boxing.

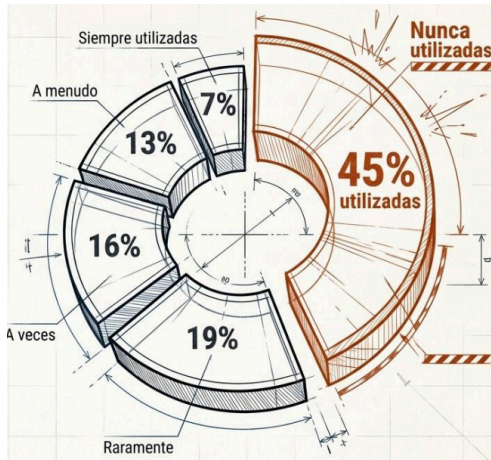
6)Gestión de productos

a)Producto de software

Un producto de Software es un artefacto que se construye para satisfacer una necesidad específica en base a ciertos requerimientos

Cuando creamos un producto debemos enfocarnos en aquello que es de valor para el cliente, sabiendo que el 7% de las características del software son siempre usadas. Esto implica, que la mayor parte del valor agregado está concentrado en ese 7%. Alrededor del

80% de las características del software que se desarrolla no se utilizan. El desafío está en encontrar ese 7%.



El mayor riesgo no es fallar técnicamente, es construir a la perfección algo que nadie quiere

El pensamiento Lean define la creación de valor como proveer beneficios reales a los clientes. Cualquier otra cosa es desperdicio.

La productividad no se mide en líneas de código, sino en averiguar sistemáticamente cuál es la cosa correcta a construir.

Escribir código no es sinónimo de crear valor. El éxito no se mide por la cantidad de funcionalidades entregadas, sino por la adopción real del cliente.

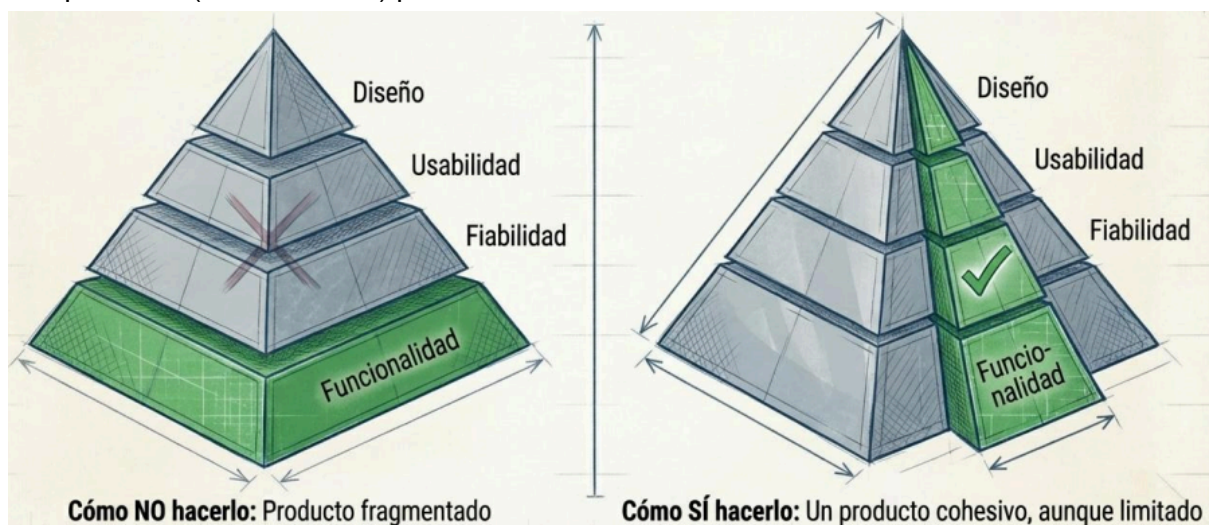
b)Producto mínimo viable

Es un concepto de Lean Startup que enfatiza el impacto del aprendizaje en el desarrollo de nuevos productos. Se define como la versión de un nuevo producto que permite a un equipo recopilar la cantidad máxima de aprendizaje validado sobre los clientes con el menor esfuerzo.

Una premisa clave detrás de la idea del MVP es que usted produce un producto real que puede ofrecer a los clientes y observar su comportamiento real con el producto o servicio. Un MVP es un experimento diseñado para confrontar la realidad lo antes posible. Requiere simplificación y evitar la sobre-construcción.

MVP efectivo

Un MVP no habla solo de diseño técnico; sirve para proveer una dosis necesaria de realidad. Debe enfocarse en la Propuesta de Valor Única (UVP) y ofrecer lo mínimo indispensable ("table stakes") para ser viable.

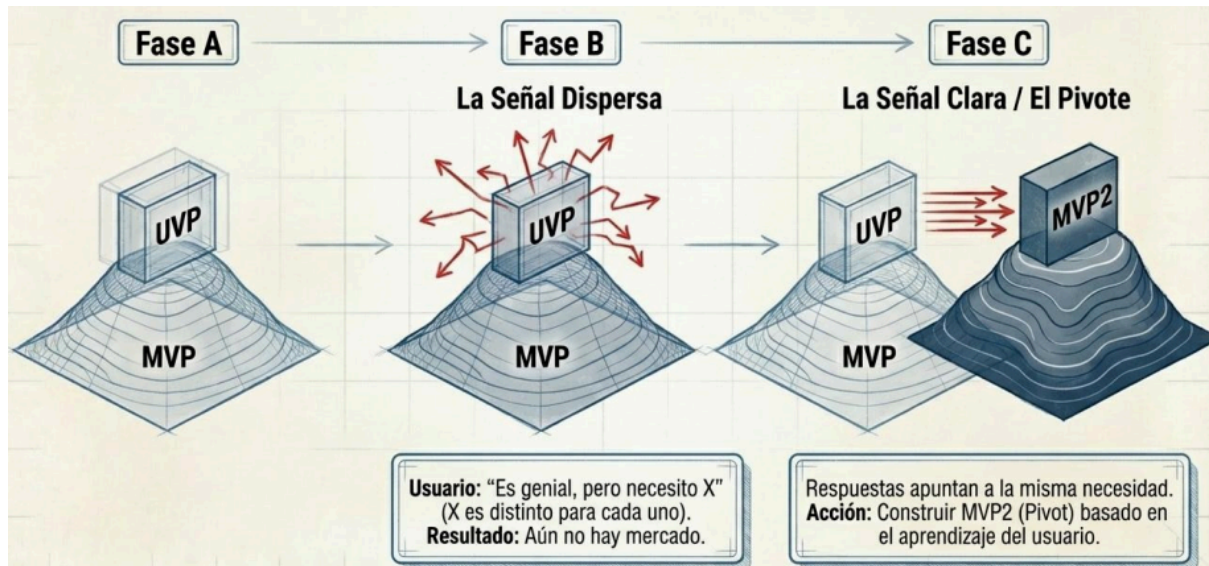


UVP

UVP es La hipótesis principal. Lo que hace que el producto o servicio sea único en el mercado.

Su relación con el MVP es que el MVP es El vehículo para probar esa hipótesis. Debe incluir funciones básicas para ser genuinamente "viable".

Si los usuarios piden requerimientos totalmente distintos, aún no has encontrado un mercado. Si todos piden la misma característica faltante, es momento de hacer un pivot (MVP 2).



Matriz de priorización

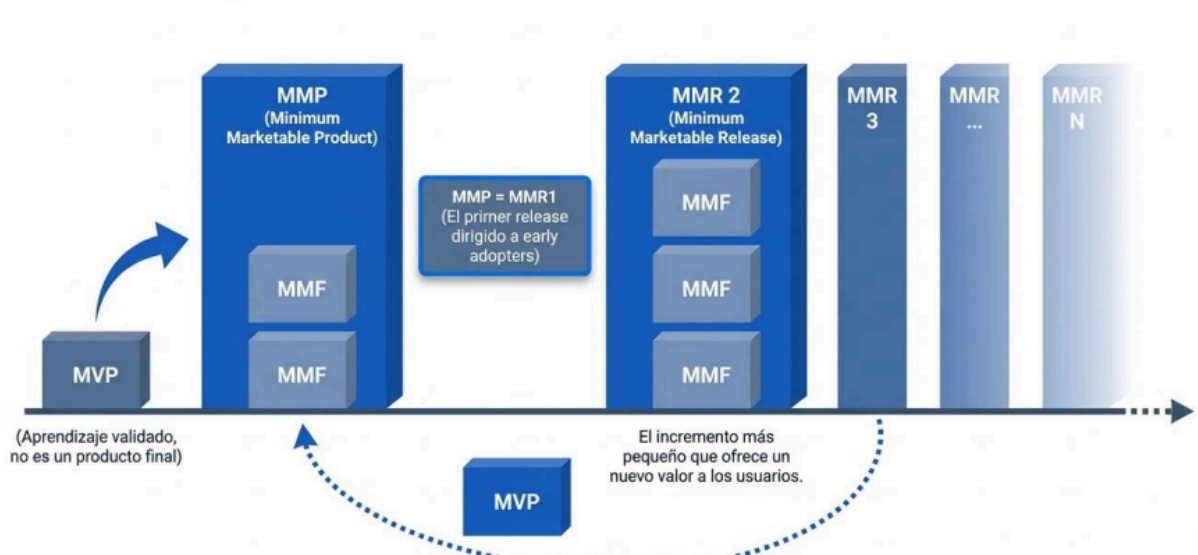


Características Mínimas Comercializables y Viables

- **MMF (Minimum Marketable Feature)**
 - Concepto: Alta certeza comercial. La pieza más pequeña de funcionalidad que puede ser liberada aportando valor inmediato y crecimiento.
 - Regla: Si al liberarlo te sientes cómodo trabajando en el próximo MMF del área, estás en el camino correcto.
- **MVF (Minimum Viable Feature)**
 - Concepto: Alta incertidumbre. Una función 'pionera' para probar un área nueva. Esencialmente una versión mini del MVP dentro de un producto existente.

Una vez logrado el encaje producto-mercado, el crecimiento se diseña combinando MMFS (para explotar certezas) y MVFs (para explorar nuevas áreas de valor).

La cronología del lanzamiento: El ecosistema de los Mínimos



MLP, Más allá de la viabilidad

Llegamos luego a un punto, donde no solo basta con las tareas, necesito tener en cuenta la experiencia.

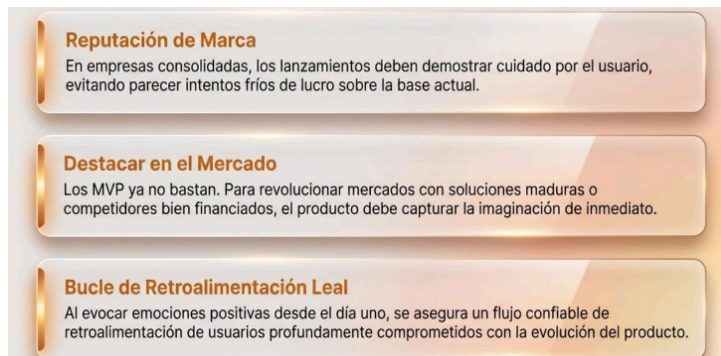
Para las organizaciones modernas, cruzar de la viabilidad técnica al amor del usuario es la grieta más difícil de sortear. Requiere un cambio radical de mentalidad: de gestionar características a curar experiencias humanas.



Por qué lo "viable" ya no es suficiente? En mercados maduros dominados por competidores bien financiados, un MVP tradicional puede parecer un intento de lucro a corto plazo.

- **Reputación de marca:** Demuestra a los usuarios que te importan sus necesidades.

- Diferenciación: Destaca donde lo funcional es un estándar mínimo.
- Feedback superior: Crea una conexión emocional temprana para retener usuarios leales.



10 bloques de construcción de la adorabilidad



c)Matriz de Diagnóstico: MVP vs MMP vs MLP

MVP (Producto Mínimo Viable)	MMP (Producto Mínimo Comercializable)	MLP (Producto Mínimo Adorable)
Enfoque: Entender necesidades Validar conceptos rápido y barato.	Enfoque: Amplio alcance / Atender a grandes bases de usuarios.	Enfoque: Diseñar para el deleite / Conexión emocional.
Foco Clave: Investigación, Código base, Pruebas de usuario.	Foco Clave: Estabilidad técnica, Lanzamiento al mercado.	Foco Clave: UI avanzada, Impacto emocional, Reducción de fricción.
Resultado: Informar el alcance del producto.	Resultado: Atracción de mercado masivo.	Resultado: Lealtad temprana y retención.

El rol del ingeniero moderno no es asegurar que el código no falle. Es asegurar que el producto resuelva el problema del usuario evocando una emoción positiva constante.



7) Proyecto de software

a) Características

- Únicos: El resultado de un proyecto será único e irreplicable para cualquier otro proyecto, por más parecido o igual sean sus elementos componentes.
- Duración limitada: Los proyectos son temporarios, cuando se alcanza el/los objetivo/s, el proyecto termina. Una línea de producción no es un proyecto.
- Orientado a objetivos: Los proyectos están dirigidos a obtener resultados y ello se refleja a través de objetivos. Los objetivos guían al proyecto y no deben ser ambiguos, y deben ser alcanzables
- Tareas interrelacionadas basadas en esfuerzos y recursos: Complejidad sistémica de los problemas.

b) Necesidad de gestionar el desarrollo de software

El software es complejo.

Aquellas herramientas que tienen a eliminar las barreras artificiales son la parte “accidental” de crear software, pero lo complejo realmente se encuentra en la esencia (complejidad, conformidad, mutabilidad, invisibilidad). Las herramientas, entonces, apuntan a los accidentes y no a la esencia.

Las ganancias pasadas en productividad vinieron de eliminar barreras artificiales. Esa era la parte fácil

El Accidente (Problemas ya resueltos)	La Esencia (Problemas incurables)
☞ Representación en lenguajes engorrosos. ☞ Restricciones severas de memoria. ☞ Tiempos largos de compilación. ☞ Procesamiento por lotes (Batch) vs. Interactivo. (Fácil de automatizar y acelerar)	1. Complejidad: Ninguna parte del software es igual a otra. 2. Conformidad: Debe interactuar con sistemas humanos arbitrarios e ilógicos. 3. Mutabilidad: La presión constante por cambiar; el software es infinitamente maleable. 4. Invisibilidad: No tiene una representación geométrica o física inherente.

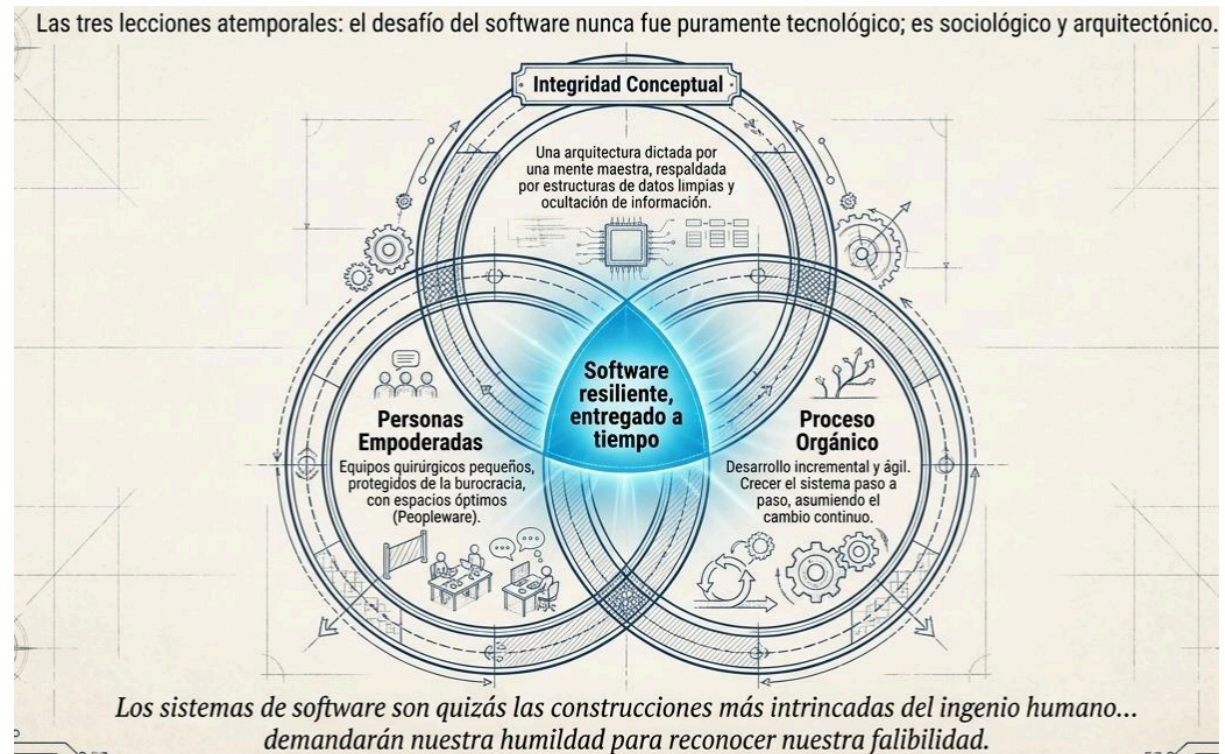
“No hay balas de plata”

No hay bala de plata para la complejidad esencial. Ningún desarrollo tecnológico por sí solo promete una mejora de 10x en productividad, fiabilidad o simplicidad.

La Ilusión Tecnológica. La Programación Orientada a Objetos, la Inteligencia Artificial o los nuevos lenguajes atacan la representación del concepto (el Accidente), no la creación del constructo mental en sí (la Esencia).

Conclusión: La parte más difícil de construir software es la especificación, diseño y prueba de los constructos conceptuales, no el trabajo manual de escribir código.

Ecosistema del desarrollo exitoso



c) Administración de proyectos (para la gestión tradicional)

Administración de proyectos es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer los requerimientos del proyecto.

Es “...tener el trabajo hecho...” en tiempo, con el presupuesto acordado y habiendo satisfecho las especificaciones o requerimientos.

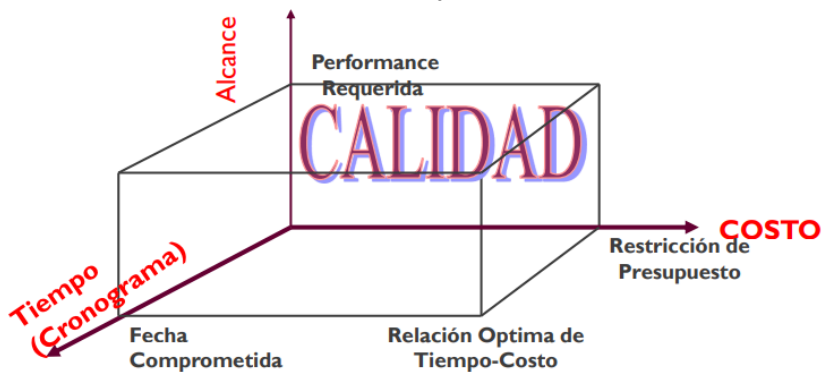
Incluye:

- Identificar los requerimientos
- Establecer objetivos claros y alcanzables
- Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados (stakeholders).

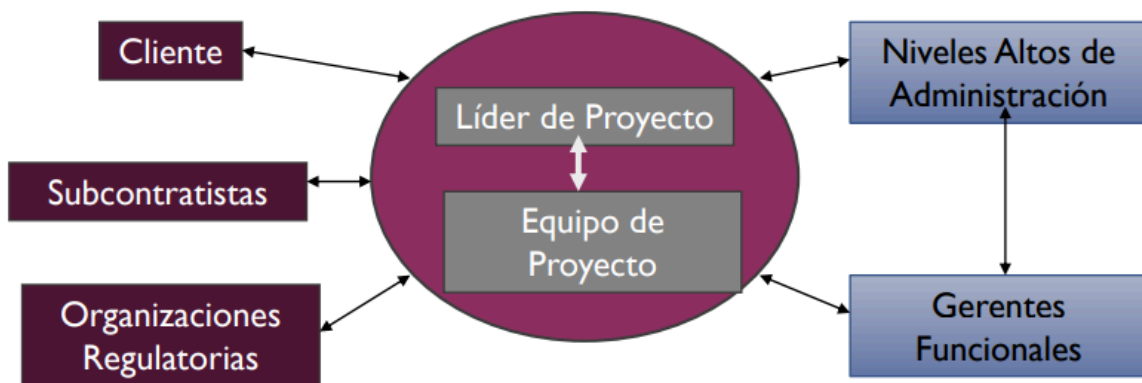
Restricción triple

- Objetivos de proyecto: que está el proyecto tratando de alcanzar?
- Tiempo: cuánto tiempo debería llevar completarlo?
- Costos: cuánto debería costar?

El balance de estos tres factores afecta directamente la calidad del proyecto, ya que un proyecto de alta calidad es aquel que entrega el producto, servicio o resultado requerido, satisfaciendo los objetivos en el tiempo estipulado y con el presupuesto planificado. Es responsabilidad del Líder de proyecto balancear estas variables.



d) Rol del líder de proyecto/equipo



Equipo de proyecto: Un grupo de personas comprometidas en alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables.

Características de un equipo de proyecto

- ✱ Diversos conocimientos y habilidades
- ✱ Posibilidad de trabajar juntos efectivamente / desarrollar sinergia
- ✱ Usualmente es un grupo pequeño
- ✱ Tienen sentido de responsabilidad como una unidad

Stakeholders Son los interesados del proyecto, incluye el equipo de proyecto, el equipo de dirección, el líder de proyecto y el patrocinador.

e) Plan de proyecto

El líder de proyecto se encarga de crear y mantener el plan de proyecto

Es la definición del tiempo, costo, cuanto, quienes, como se transita, etc

El plan de proyecto documenta:

- ¿Qué es lo que hacemos?
- ¿Cuándo lo hacemos?
- ¿Cómo lo hacemos?

- ¿Quién lo va a hacer?

La planificación de un proyecto implica

- **Definición del alcance**

- Alcance del Producto: Son todas las características que pueden incluirse en un producto o servicio.
- Alcance del Proyecto: Es todo el trabajo y solo el trabajo que debe hacerse para entregar el producto o servicio con todas las características y funciones especificadas
- Cómo se miden:
 - El cumplimiento del Alcance del Proyecto: Se mide contra el Plan de Proyecto (o Plan de Desarrollo de Software).
 - El cumplimiento del Alcance del Producto: Se mide contra la Especificación de Requerimientos.

- **Definir un ciclo de vida**

- El ciclo de vida del proyecto es lo que define cuánto de cada tarea se debe hacer y en qué momento del desarrollo del proyecto hacerlo.
- En metodologías tradicionales, se permite elegir cualquier tipo de ciclo de vida (cascada, iterativo e incremental, espiral, etc.), en cambio en metodologías ágiles esto no es posible (si o si tiene que ser iterativo e incremental, y no va a haber un plan; el plan solo aplica a procesos definidos, no a empíricos).

- **Estimaciones de software**

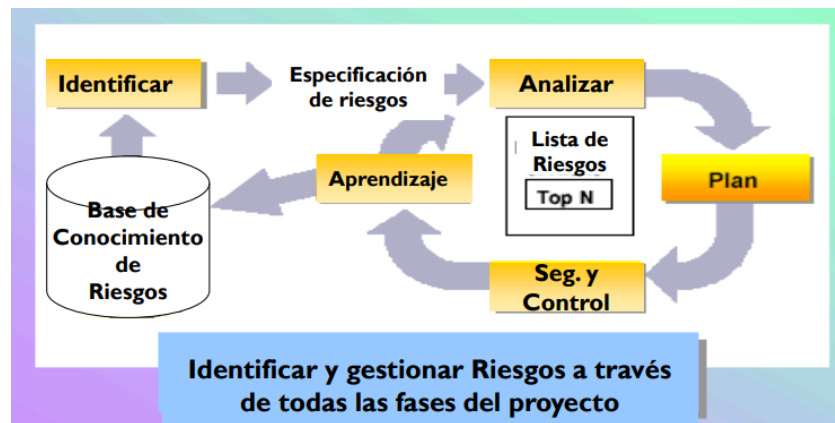
- En un proyecto de desarrollo de software, se trata de estimar para predecir el valor de un elemento relacionado con el proyecto o con el producto, por ejemplo, el tiempo que va a llevar, qué costo va a tener el sistema, cuántas personas necesito para realizar el desarrollo, etc. En un enfoque tradicional, esta estimación de valores es realizada únicamente por el líder de proyecto. En la metodología tradicional, existe un orden definido y recomendado para realizar las estimaciones:
 - Tamaño: para estimar el tamaño de un proyecto se utilizan los requerimientos tomados en la etapa de requisitos.
 - Esfuerzo: Refiere a la cantidad de horas persona lineales haciendo una actividad por vez, dentro del desarrollo del producto.
 - Calendario: la estimación del calendario del proyecto permite responder al cuándo se van a realizar las tareas definidas.
 - Costo: de forma similar a la estimación de la duración del proyecto, la estimación de costos depende directamente del tamaño y del esfuerzo que se necesite para el desarrollo del software, por eso es lo último que se estima
 - Recursos Críticos

- **Gestión de riesgos**

- Riesgo: Problema esperando para suceder. Evento que podría comprometer el éxito del proyecto
- ¿Cómo lo gestionamos?
 - Identificación de riesgos: se identifican los riesgos que suponen una mayor amenaza al proceso de ingeniería de software, al software a

desarrollar o a la organización que lo desarrolla (al recibir los requerimientos).

- Análisis de riesgos: para cada riesgo identificado se realiza un juicio acerca de la probabilidad y del impacto.
 - Planeación del riesgo: para cada riesgo analizado, se definen estrategias para manejarlos. Estas estrategias consisten en considerar acciones a tomar para minimizar o evitar el impacto sobre el proyecto como consecuencia de la ocurrencia de la amenaza que representa el riesgo.
 - Monitorización del riesgo y control: proceso que permite determinar que las suposiciones acerca de los riesgos de producto, proceso y organización no han cambiado.
- Impacto: si este riesgo ocurre, ¿qué pasa? → cuantificamos el impacto
 - mitigar probabilidad de ocurrencia, el impacto o ambos (depende del riesgo, en algunos casos solo se puede mitigar uno de estos aspectos)

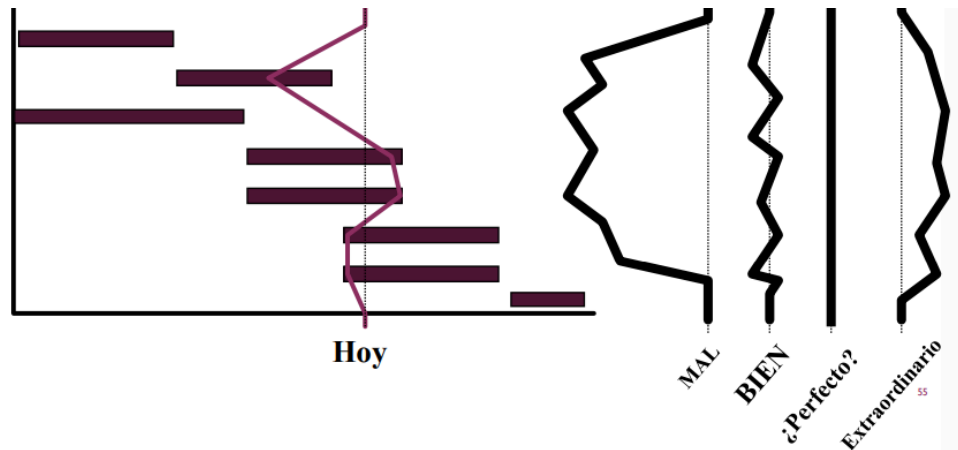


- **Métricas de software**

- Métricas de proceso: Medir en muchos proyectos el mismo proceso. No nos importa un proyecto en particular, nos importa en términos de organización como trabajamos.
- Métricas de proyecto: Medir lo que nos interese del proyecto para saber si vamos bien con lo que venimos haciendo o si debemos aplicar acciones correctivas
 - Las métricas del proyecto se consolidan para crear métricas de proceso que sean públicas para toda la organización del software.
 - Básicas: Tamaño del producto • Esfuerzo • Calendario (tiempo) • Defectos • Costos
- Métricas básicas:
 - Proyecto: esfuerzo, tiempo
 - Proceso: las mismas con distinto foco
 - Producto: Tamaño producto, defectos
- Algunas cuestiones:
 - No utilizamos métricas para medir a las personas
 - Tomar métricas no debe ser complicado
 - Las métricas deben dar utilidad (tienen que darnos algún valor)

- **Monitoreo y control**

- Comparar lo planificado con lo real



-
- Lo planificado es teórico, no hay un “perfecto”
- Se realizan acciones correctivas y modificaciones al plan

Factores para el éxito del proyecto:

- Monitoreo & Feedback
- Tener una misión/objetivo claro
- Comunicación

Causas de fracasos en proyectos:

- Fallas al definir el problema
- Planificar basado en datos insuficientes
- La planificación la hizo el grupo de planificaciones
- No hay seguimiento del plan de proyecto
- Plan de proyecto pobre en detalles
- Planificación de recursos inadecuada
- Las estimaciones se basaron en “supuestos” sin consultar datos históricos
- Nadie estaba a cargo

Vinculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.



El **proceso** define cómo se va a desarrollar el software: actividades, métodos y prácticas. No existe un proceso ideal para todos los proyectos, por lo que debe **adaptarse al proyecto** según el tipo y complejidad del producto.

El **proyecto** incorpora **personas, roles y herramientas** que siguen el proceso elegido para desarrollar el producto. Primero se definen los **objetivos** (qué se quiere lograr) y el **ámbito del producto** (qué funciones y comportamientos tendrá).

El **producto** es el resultado final del proyecto, obtenido mediante la aplicación del proceso.

En resumen:

Proceso → define cómo trabajar.

Proyecto → aplica y adapta el proceso con personas y herramientas.

Producto → resultado obtenido.

Además, se debe elegir un modelo de proceso (**secuencial, iterativo o recursivo**) y adaptarlo a las necesidades del proyecto. Las **personas son fundamentales**, porque el desarrollo de software es una actividad principalmente humana.

8) SCRUM 2026

(repaso)

Procesos empíricos

-Ágil está basado en un proceso de control empírico

-Si tenemos entornos de valores cambiantes, el empirismo nos ayuda

-Los principios no son prácticas: una práctica concreta me da un marco de trabajo, el principio es más filosófico

a) Scrum: El framework

Scrum es un marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos.

Es un framework, no una metodología:

- metodología: define el paso a paso
- framework: es un marco de trabajo, define ciertos aspectos

Scrum 2026

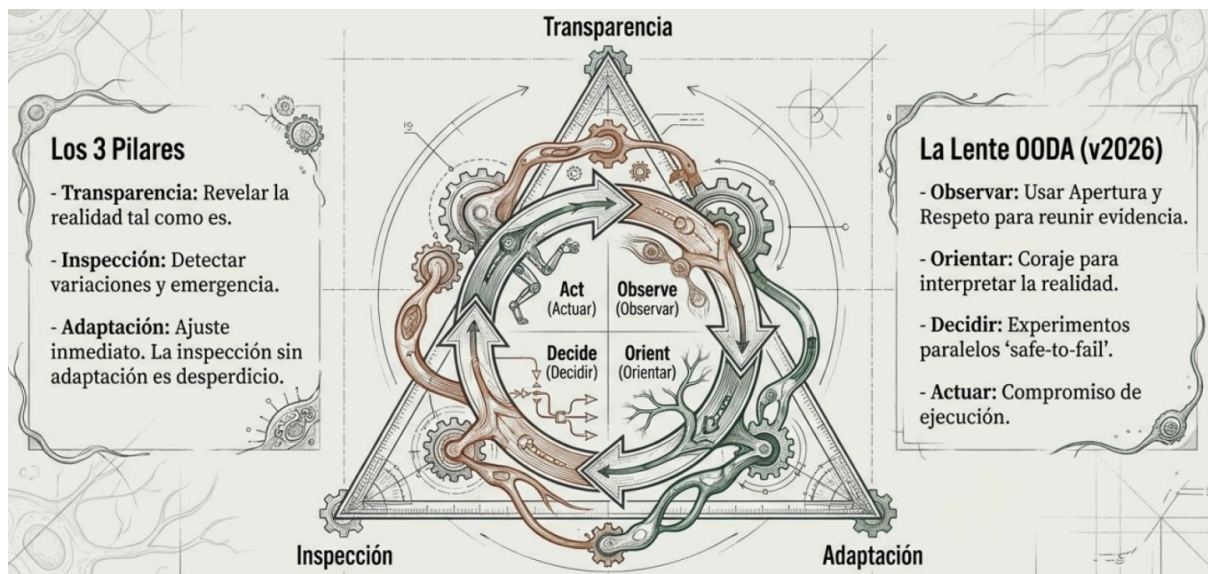
Incluye a la IA como agente (nuevo manifiesto pero incluyendo a la ia)

b) Principios

- Control de proceso empírico: Aprender de la experiencia. Scrum se basa en el empirismo y el pensamiento Lean. El empirismo afirma que el conocimiento proviene de la experiencia y la toma de decisiones basadas en lo que se observa. El pensamiento Lean reduce los desperdicios y se centra en lo esencial.
- Auto-organización: el equipo define las tareas, se autogestiona
- Colaboración: entre los integrantes
- Priorización basada en valor: En el product backlog, tiene más prioridad lo que tiene más valor
- Time boxing

- Desarrollo interactivo: funciona con ciclo de vida iterativo e incremental para optimizar la previsibilidad y controlar el riesgo.

c)Empirismo y el ciclo OODA



La lente OODA (del nuevo enfoque 2026) ordena los valores

- Observar: Usar apertura y respeto para reunir evidencia
- Orientar: coraje para interpretar la realidad
- Decidir: Experimentos paralelos "Safe to fail"
- Actuar: compromiso de ejecución

Pilares de Scrum

- **Transparencia:** El proceso y el trabajo emergentes deben ser visibles para aquellos que realizan el trabajo, así como para los que reciben el trabajo. Con Scrum, las decisiones importantes se basan en el estado percibido de sus tres artefactos formales. Los artefactos que tienen poca transparencia pueden conducir a decisiones que disminuyen el valor y aumentan el riesgo. La transparencia permite la inspección. La inspección sin transparencia genera engaños y desperdicios.
- **Inspección:** Los artefactos de Scrum y el progreso hacia objetivos acordados deben ser inspeccionados con frecuencia y diligentemente para detectar varianzas o problemas potencialmente indeseables. Para ayudar con la inspección, Scrum proporciona cadencia en forma de sus cinco eventos. La inspección permite la adaptación. La inspección sin adaptación se considera inútil. Los eventos de Scrum están diseñados para provocar cambios.
- **Adaptación:** Si algún aspecto de un proceso se desvía fuera de los límites aceptables o si el producto resultante es inaceptable, el proceso que se está aplicando o los materiales que se producen deben ajustarse. El ajuste debe realizarse lo antes posible para minimizar la desviación adicional.

d)Valores de Scrum

El uso exitoso de Scrum depende de que las personas sean más competentes en vivir cinco valores: Compromiso, Enfoque, Apertura, Respeto y Coraje

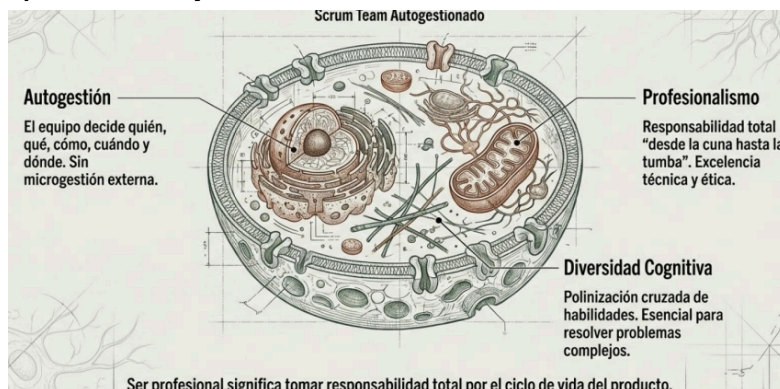
- ✿ El equipo de Scrum se compromete a lograr sus objetivos y apoyarse mutuamente

- ✿ Su enfoque principal es el trabajo del Sprint para hacer el mejor progreso posible hacia estos objetivos.
- ✿ El equipo de Scrum y sus partes interesadas están abiertos sobre el trabajo y los desafíos
- ✿ Los miembros del equipo de Scrum se respetan mutuamente para ser personas capaces e independientes, y son respetados como tales por las personas con las que trabajan.
- ✿ Los miembros del equipo de Scrum tienen el valor de hacer lo correcto y de trabajar en problemas complejos.
- ✿ Estos valores dan dirección al equipo de Scrum con respecto a su trabajo, acciones y comportamiento. Las decisiones que se toman, las medidas tomadas y la forma en que se utiliza Scrum deben reforzar estos valores, no disminuirlos o socavarlos. Los miembros del equipo de Scrum aprenden y exploran los valores mientras trabajan con los eventos y artefactos de Scrum. Cuando estos valores son asimilados por el equipo de Scrum y las personas con las que trabajan, los pilares empíricos de Scrum de transparencia, inspección y adaptación cobran vida construyendo confianza.



La confianza es la moneda de cambio. Sin Coraje para decir "no" y Respeto para criticar ideas (no personas), el empirismo falla.

e)La célula profesional



Equipo que lleva adelante scrum que se autogestiona, diversamente cognitivo y tiene profesionalismo (todo el equipo es igual de responsable)

Ecosistema extendido: Stakeholders y Supporters



Stakeholders (Interesados)

-Clientes, usuarios, decisores ('Choosers'), e incluso entidades no humanas (la ley, IA).

-Rol: Proveen feedback, requisitos y validan el valor.

Supporters (Agentes de Cambio)

-Líderes y gerentes que apoyan activamente la adopción de Scrum. Scrum Team

-Rol: Eliminan barreras organizacionales, proveen recursos y 'cobertura aérea' política. Diferenciación: Si un manager no empodera al equipo ni remueve impedimentos, no es un Supporter.

El equipo no vive en el vacío; necesita una coalición de Supporters para sobrevivir a la inercia organizacional.

f)Roles

Roles en la guía extendida

Rol	Naturaleza	Responsabilidad Principal
Product Owner	Humano	Maximizar el valor a largo plazo; gestionar el Product Backlog y la estrategia del producto.
Product Developer	Humano o Automatizado	Crear incrementos utilizables; adherirse a la Definición de Salida Terminada; autogestionar el plan del Sprint.
Scrum Master	Humano	Agente de cambio; guía la efectividad del equipo y la organización en la adopción de Scrum.
Stakeholder	Humano/Entidad/IA	Entidades interesadas o afectadas por el producto (clientes, usuarios, patrocinadores, legisladores).
Supporter	Stakeholder específico	Coalición poderosa que elimina impedimentos organizacionales y fomenta un clima para la autogestión.

Roles Scrum team

La unidad fundamental de Scrum es un pequeño equipo de personas, un equipo Scrum. El equipo Scrum consta de un Scrum Master, un propietario de producto (Product Owner) y desarrolladores. Dentro de un equipo de Scrum, no hay sub-equipos ni jerarquías. Es una unidad cohesionada de profesionales enfocada en un objetivo a la vez, el objetivo del Producto.

- **Product owner:** El Propietario del Producto es responsable de maximizar el valor del producto resultante del trabajo del equipo de Scrum. La forma en que esto se hace puede variar ampliamente entre organizaciones, equipos Scrum e individuos.

El Propietario del Producto también es responsable de la gestión eficaz de la pila del producto (Product Backlog). El “Qué” y el “Por qué”

- El Propietario del Producto es una persona, no un comité.
- **SCRUM master:** Líder servicial y agente de cambio (Change Agent). Gestiona la actividad del entorno y remueve impedimentos organizacionales. La efectividad
 - El Scrum Master es responsable de establecer Scrum tal como se define en la Guía de Scrum.
 - No es un secretario ni un administrador de Jira
- **Product developers:** Crean el incremento “Done” (terminado). Gestionan el plan táctico (Sprint Backload). Incluye humanos y agentes automatizados. El “Cómo” y la calidad

El Scrum Team: Autogestión e Inteligencia Aumentada

- Un equipo pequeño, cognitivamente diverso y empoderado que posee todas las habilidades para crear valor.
- Autogestión (Self-Managing) El equipo decide quién hace qué, cómo y cuándo. No hay jerarquías, solo responsabilidades compartidas.
- Al como 'Compañero de Equipo': La Inteligencia Artificial se integra en el descubrimiento y entrega.
 - Human in the loop: La IA aumenta la capacidad, pero la responsabilidad (accountability) permanece 100% en los humanos.
 - Uso ético: La IA ayuda a validar hipótesis y reducir sesgos, no a reemplazar el juicio profesional.

Liderazgo y cambio - Genchi Genbutsu

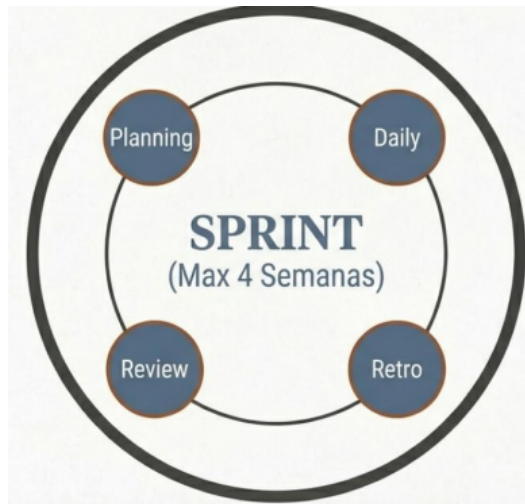
- Liderazgo para la resiliencia
 - Es una conducta, no un cargo
 - Crear claridad y un entorno seguro para la emergencia
 - Genchi Genbutsu: ve, escucha y entiende. Decisiones basadas en hechos reales, no reportes distantes
- El cambio es emergente
 - No se puede forzar un destino fijo
 - Se guía una dirección de viaje
 - Los líderes actúan como catalizadores

g)El Sprint

Los sprints son el latido del corazón de Scrum, donde las ideas se convierten en valor.

Son eventos de longitud fija de un mes o menos para crear consistencia. Un nuevo Sprint comienza inmediatamente después de la conclusión del Sprint anterior.

Todo el trabajo necesario para alcanzar el objetivo del producto, incluyendo la Planificación (Sprint Planning), Daily Scrums, Revisión del Sprint (Sprint Review) y la Retrospectiva (Sprint Retrospective), ocurren dentro del Sprints.



Contenedor de consistencia

- **Cadencia:** Crea hábito y reduce la complejidad cognitiva. El sprint tiene un timebox de máximo cuatro semanas, definimos cadencia (tiempo del timebox). Mientras más pequeña la cadencia, más controlable.
 - Cadencia fija: Que el cliente tenga predicibilidad, si le decimos cuatro semanas, sabe que cada cuatro semanas le vamos a hacer una entrega
- **Control de riesgo:** Sprint más cortos equivale a menor costo de error. Cada sprint es una oportunidad para pivotar

Inmutabilidad del objetivo: El sprint tiene un objetivo (sprint goal) que es inamovible, el alcance (scope) es negociable entre PO y developers si ocurre aprendizaje. Es decir, si con aprendizaje cambió la forma en la que haría algo pero el objetivo se mantiene, el sprint continua y con el mismo objetivo.

No hay sprint de Hardening: La calidad (output done) se construye en cada sprint (no se hace sprints para la corrección de algo que se hizo en un sprint anterior)

Eventos del sprint:

- ★ **Daily scrum:** El propósito del Daily Scrum es inspeccionar el progreso hacia el Objetivo Sprint y adaptar el Sprint Backlog según sea necesario, ajustando el próximo trabajo planeado. El Daily Scrum es un evento de 15 minutos (máximo) para los desarrolladores del equipo de Scrum. Para reducir la complejidad, se lleva a cabo al mismo tiempo y lugar todos los días laborables del Sprint.
- ★ **Sprint review:** El propósito de la revisión del Sprint es inspeccionar el resultado del Sprint (outcome) y determinar futuras adaptaciones. El equipo de Scrum presenta los resultados de su trabajo a las partes interesadas clave y se discute el progreso hacia el Objetivo de Producto. Durante el evento, el equipo de Scrum y las partes interesadas revisan lo que se logró en el Sprint y lo que ha cambiado en su entorno. En base a esta información, los asistentes colaboran en qué hacer a continuación
- ★ **Sprint retrospective:** El propósito de la retrospectiva Sprint es planificar formas de aumentar la calidad y la eficacia. El equipo de Scrum inspecciona cómo fue el último Sprint con respecto a individuos, interacciones, procesos, herramientas y su definición de Hecho. Los elementos inspeccionados a menudo varían según el

dominio del trabajo. Las suposiciones que los desviaron se identifican y se exploran sus orígenes. El equipo de Scrum analiza qué fue bien durante el Sprint, qué problemas encontró y cómo esos problemas fueron (o no fueron) resueltos. El equipo de Scrum identifica los cambios más útiles para mejorar su eficacia.

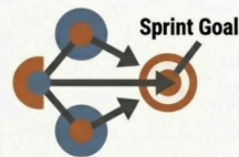
- ★ **Planning:** El Sprint Planning inicia el Sprint estableciendo el trabajo que se realizará para el mismo. Este plan resultante es creado por el trabajo colaborativo de todo el equipo de Scrum. Se toma el product backlog y definimos que us se incluyen en el sprint

Planificación y Sincronización: Inspección Diaria

Sprint Planning (El Diseño Táctico)

Define tres aspectos críticos:

1. **Por Qué:** El Sprint Goal.
2. **Qué:** Items seleccionados del Backlog.
3. **Cómo:** Plan técnico de ejecución.



Nota: El equipo no debe sobrecargarse; planificar holgura (**slack**) para manejar la incertidumbre.

Daily Scrum (Navegación Táctica)

Una replanificación de 15 minutos de los Developers para los Developers.

- **NO ES:** Un reporte de estado para el Scrum Master.
- **ES:** Sincronización táctica.
- **FOCO:** Progreso hacia el Sprint Goal y eliminación de bloqueos inmediatos.



Revisión y Mejora: Cerrando el Ciclo de Aprendizaje

Icono que muestra una lupa con un ojo en el centro, representando la inspección.

Sprint Review (Validación de Valor)

- Sesión de trabajo colaborativa, no una presentación.
- Inspección del Incremento real.
- Validación contra la **Definition of Outcome Done**:
¿Resolvimos el problema del usuario?

Icono que muestra una flecha circular con una llave inglesa, representando la mejora y el ciclo de aprendizaje.

Sprint Retrospective

- Foco en la calidad del equipo, procesos y herramientas.
- Búsqueda de la "Quality Without a Name" (coherencia y vitalidad).
- Búsqueda de la "Quality Without a Name" (coherencia y vitalidad).
- Las mejoras deben ejecutarse lo antes posible, incluso dentro del próximo Sprint.

Timebox en Scrum

- 🕒 Sprint: 1 mes o menos
- 🕒 Sprint Planning: 8 horas máximo para un Sprint de 1 mes
- 🕒 Daily Meeting: 15 minutos
- 🕒 Sprint Review: 4 horas máximo para un Sprint de 1 mes
- 🕒 Sprint Retrospective: 3 horas máximo para un Sprint de 1 mes
- 🕒 Refinamiento del PB: 10% del tiempo de Sprint

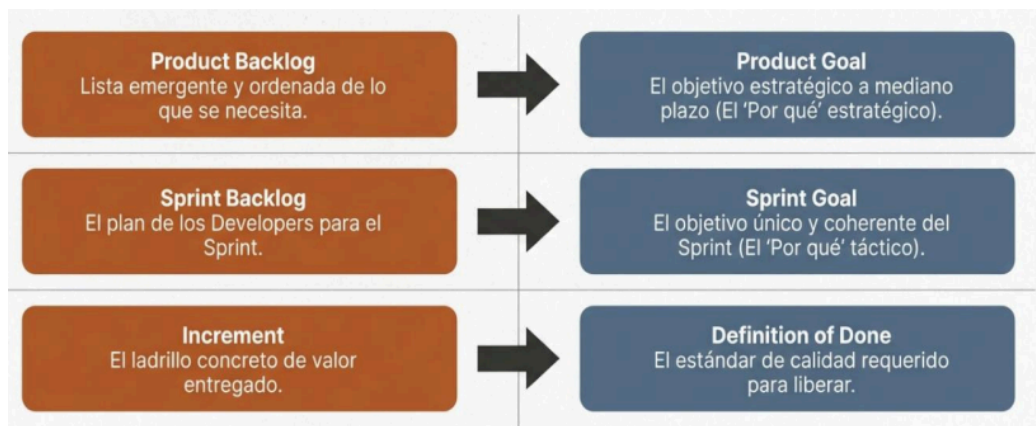
h) De salida a impacto

- Output Done
 - Estándar de calidad técnica, incremento lo utilizable. Pregunta: ¿Lo construimos correctamente?
- Outcome Done
 - Evidencia observable de valor. Cambio de comportamiento en el usuario. Pregunta: ¿Construimos lo correcto? ¿Generamos valor?

La Doble Definición de "Hecho": Output vs. Outcome

Definition of OUTPUT Done (Calidad Técnica)	Definition of OUTCOME Done (Valor de Negocio)
Enfoque: ¿Construimos el producto correctamente?	Enfoque: ¿Construimos el producto correcto?
Criterios: • Código limpio y testado • Integrado y Documentado • Cumple estándares	Criterios: • Cambio en comportamiento del usuario • Impacto en el negocio / ROI • Validación de hipótesis
Momento: Debe cumplirse dentro del Sprint para considerar el Incremento.	Momento: Medible a menudo después del lanzamiento (Lagging indicator).

i) Artefactos y compromisos



1. **Product Backlog:** Lista ordenada y dinámica de todo el trabajo necesario para mejorar el producto. Se refina continuamente, agregando detalles y dividiendo los elementos cuando es necesario. **Product Goal:** Es el objetivo a largo plazo del equipo Scrum. Describe el estado futuro que se quiere alcanzar con el producto y guía el trabajo del Product Backlog.
2. **Sprint Backlog:** Es el plan de trabajo del Sprint. Incluye el objetivo del Sprint, los elementos seleccionados del Product Backlog y el plan para realizarlos. Se actualiza durante el Sprint. **Sprint Goal:** Es el único objetivo del Sprint. Da enfoque al equipo y permite flexibilidad sobre qué trabajo realizar, siempre que se mantenga el objetivo.

3. **Incremento:** Es un avance concreto y utilizable hacia el Product Goal. Debe funcionar correctamente y cumplir con la Definición de Hecho. Puede haber varios incrementos en un mismo Sprint. **Definition of Done:** Es la descripción de las condiciones que debe cumplir un trabajo para considerarse terminado. Garantiza que todos tengan el mismo criterio de calidad.

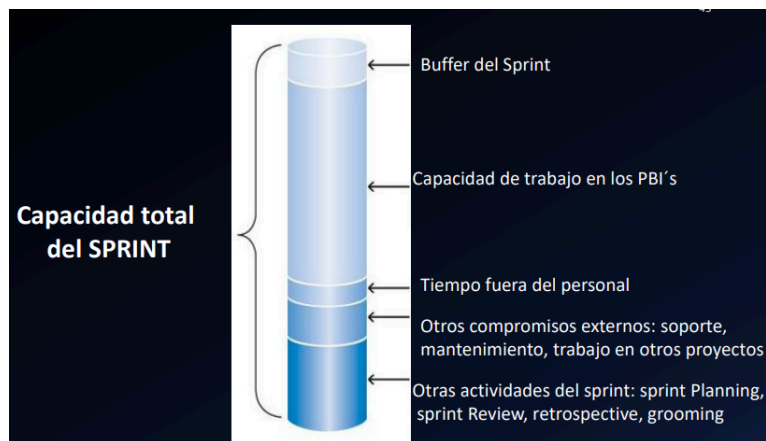
j)Scrum 2026: evolución y expansión del marco de trabajo



- Ecosistema de Roles Ampliado
 - Supporter
 - Stakeholder
 - Scrum Team
 - Scrum Master (Humano)
 - Product Owner (Humano)
 - Product Developers (H + IA)
- Reuniones
 - Sprint Planning
 - Daily Scrum
 - Sprint Review
 - Sprint Retrospective
 - Refinamiento del PB (actividad continua)
- artefactos
 - Product Backlog → Objetivo del Producto
 - Sprint Backlog → Objetivo del Sprint
 - Incremento del Producto → Definition of Done
 - → Output (Salida): Compromiso Técnico
 - → Outcome (Resultado): Compromiso de Valor

	Definition of Ready	Definition of Output Done	Definition of Outcome Done
Estatus en el SGEP	Cualidad emergente (Refinement).	Compromiso formal.	Compromiso formal.
Artefacto Asociado	Product Backlog Item (PBI).	Increment.	Product.
Responsabilidad y Decisión	Scrum Team colaborativo (Devs estiman).	Developers (Accountability colectiva).	Product Owner (Final say).
Métrica Típica	Días / Capacidad de comprensión.	Tests, Seguridad, Estándares técnicos.	Telemetría, Comportamiento de usuario, ROI.

k)Capacidad del equipo en un Sprint

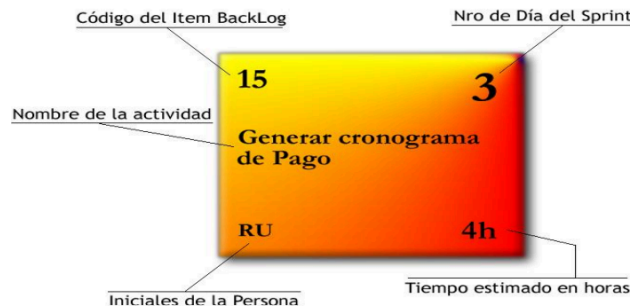


La capacidad es una métrica que utiliza SCRUM para determinar cuánta cantidad de trabajo puede comprometer un equipo para un determinado Sprint. La capacidad se estima y se mide. Una de las cosas que se hace en el Sprint Planning es determinar la capacidad del equipo, y entonces, teniendo en cuenta la capacidad, se contrasta en cuántas US del Product Backlog se pueden incorporar en el Sprint Backlog.

Persona	Días disponibles (sin tiempo personal)	Días para otras actividades Scrum	Horas por día	Horas de Esfuerzo disponibles
Jorge	10	2	4-7	32-56
Betty	8	2	5-6	30-36
Simón	8	2	4-6	24-36
Pedro	9	2	2-3	14-21
Raúl	10	2	5-6	40-48
Total				140-197

I) Herramientas de Scrum

Task board

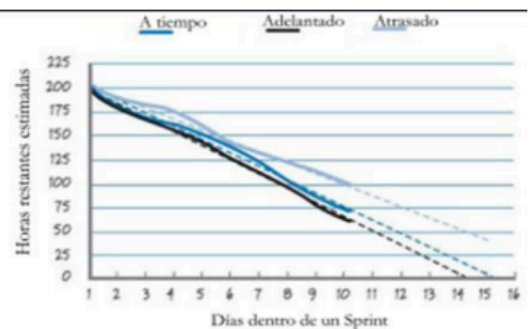


Story	To Do	In Process	To Verify	Done
As a user, I... 8 points	Code the... 9 Code the... 2 Test the... 8	Code the... DC 4 Test the... SC 8	Test the... SC 6	Code the... DC 8 Test the... SC 8 Test the... SC 6
As a user, I... 5 points	Code the... 8 Code the... 4	Code the... DC 8		Test the... SC 8 Test the... SC 6

Gráficos del Backlog

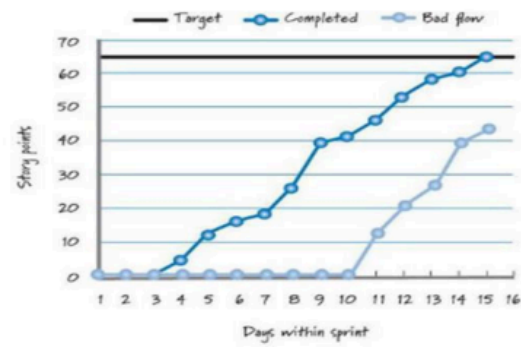
- Sprint burndown Charts**

Visualiza con una curva descendente cuánto trabajo queda para terminar. En el eje de las abscisas se coloca el tiempo que va desde el inicio del sprint. En el eje de las ordenadas se coloca los puntos de historia a realizar en el sprint.



-
- **Sprint Burn-Up Chart**

Visualiza con una curva ascendente cuánto trabajo se ha completado a lo largo de la release, es decir que visualiza los puntos de historia terminados en cada sprint.



●

Gráficos de seguimiento del producto: Release Burnup chart, product Burndown, Story Points Delivered